

Table of Contents

1	POPolitics – Personas	12
1.1	Introduction	12
1.2	Persona 1 — Le Citoyen engagé	13
1.2.1	👤 Profil	13
1.2.2	Biographie.....	13
1.2.3	Objectifs	13
1.2.4	Frustrations	13
1.2.5	Comportement numérique.....	14
1.2.6	Scénarios d’usage sur POPolitics	14
1.2.7	Citation représentative	14
1.3	Persona 2 — Le Journaliste / Expert politique.....	14
1.3.1	🗣️ Profil	14
1.3.2	Biographie.....	15
1.3.3	Objectifs	15
1.3.4	Frustrations	15
1.3.5	Comportement numérique.....	15
1.3.6	Scénarios d’usage sur POPolitics	16
1.3.7	Citation représentative	16
1.4	Persona 3 — L’Administrateur plateforme	16
1.4.1	⚙️ Profil	16
1.4.2	Biographie.....	16
1.4.3	Objectifs	17
1.4.4	Frustrations	17
1.4.5	Comportement numérique.....	17
1.4.6	Scénarios d’usage sur POPolitics	17
1.4.7	Citation représentative	18
1.5	Récapitulatif	18
2	POPolitics – Choix techniques.....	18
2.1	1. Vision générale	18
2.2	2. Objectifs fonctionnels de l’application	19

2.2.1	2.1 Objectifs principaux	19
2.2.2	2.2 Contraintes clés.....	19
2.3	3. Vision globale de l’architecture.....	19
2.3.1	3.1 Approche retenue	19
2.3.2	3.2 Motivations.....	19
2.4	4. Sources de données & ingestion	19
2.4.1	4.1 Sources exploitées.....	19
2.4.2	4.2 Caractéristiques	20
2.4.3	4.3 Choix.....	20
2.5	5. Landing Zone & Data Lake	20
2.5.1	5.1 Landing Zone – Data Lake	20
2.5.2	5.2 Organisation en 3 niveaux.....	20
2.5.3	5.3 Avantages.....	20
2.6	6. Orchestration des pipelines ETL.....	20
2.6.1	6.1 Outil : Kestra	20
2.6.2	6.2 Pourquoi Kestra ?	20
2.7	7. Stockage analytique : Datamarts	21
2.7.1	7.1 Outil : PostgreSQL	21
2.7.2	7.2 Raisons du choix	21
2.8	8. Couche IA & Model Training	21
2.8.1	8.1 Socle IA	21
2.8.2	8.2 Cas d’usage.....	21
2.8.3	8.3 Intégration	21
2.9	9. Backend – Django	22
2.9.1	9.1 Découpage en services.....	22
2.9.2	9.2 Communication	22
2.9.3	9.3 Objectifs.....	22
2.10	10. Authentification & gestion des utilisateurs	22
2.10.1	10.1 Base d’authentification dédiée.....	22
2.10.2	10.2 Flux	22
2.10.3	10.3 Objectifs.....	22
2.11	11. Frontend – Next.js	22
2.11.1	11.1 Architecture	22

2.11.2	11.2 Motivations	22
2.12	12. Synthèse	23
3	POPolitics – Priorisation MoSCoW	23
3.1	Méthode	23
3.2	1. Must have.....	23
3.2.1	1.1 Données & ETL (plateforme minimale)	23
3.2.2	1.2 Backend & services	23
3.2.3	1.3 Frontend (MVP Portail collectif)	24
3.2.4	1.4 Qualité, sécurité & documentation	24
3.3	2. Should have.....	24
3.3.1	2.1 Données & analytique	24
3.3.2	2.2 Couche IA.....	24
3.3.3	2.3 Frontend (tendances & page personnelle v1).....	24
3.3.4	2.4 Opérations & outillage	25
3.4	3. Could have	25
3.4.1	3.1 Données & IA	25
3.4.2	3.2 Frontend & UX.....	25
3.4.3	3.3 Plateforme & DevOps	25
3.5	4. Won't have (for now)	25
4	POPolitics – Exigences fonctionnelles et non fonctionnelles liées au WBS.....	26
4.1	1. Portail collectif (WBS – Fonctionnalité principale)	26
4.1.1	1.1 Description.....	26
4.1.2	1.2 Exigences fonctionnelles	26
4.1.3	1.3 Exigences non fonctionnelles.....	26
4.1.4	1.4 Critères d'acceptation (exemples).....	27
4.2	2. Page personnelle & notifications (WBS – Fonctionnalité utilisateur).....	27
4.2.1	2.1 Description.....	27
4.2.2	2.2 Exigences fonctionnelles	27
4.2.3	2.3 Exigences non fonctionnelles.....	27
4.2.4	2.4 Critères d'acceptation.....	27
4.3	3. Plateforme Data & ETL (WBS – Infrastructure data)	27
4.3.1	3.1 Description.....	27
4.3.2	3.2 Exigences fonctionnelles	28

4.3.3	3.3 Exigences non fonctionnelles.....	28
4.3.4	3.4 Critères d'acceptation.....	28
4.4	4. Datamarts & analytique (WBS – Stockage analytique).....	28
4.4.1	4.1 Description.....	28
4.4.2	4.2 Exigences fonctionnelles	28
4.4.3	4.3 Exigences non fonctionnelles.....	28
4.4.4	4.4 Critères d'acceptation.....	28
4.5	5. Couche IA & modèles (WBS – IA Service)	29
4.5.1	5.1 Description.....	29
4.5.2	5.2 Exigences fonctionnelles	29
4.5.3	5.3 Exigences non fonctionnelles.....	29
4.5.4	5.4 Critères d'acceptation.....	29
4.6	6. Authentification & gestion des utilisateurs (WBS – Auth Service)	29
4.6.1	6.1 Description.....	29
4.6.2	6.2 Exigences fonctionnelles	29
4.6.3	6.3 Exigences non fonctionnelles.....	29
4.6.4	6.4 Critères d'acceptation.....	29
4.7	7. Qualité, monitoring & opérations (WBS – Support).....	30
4.7.1	7.1 Description.....	30
4.7.2	7.2 Exigences fonctionnelles	30
4.7.3	7.3 Exigences non fonctionnelles.....	30
5	POPolitics – Definition of Done (DoD).....	30
5.1	1. Objectif	30
5.2	2. Critères généraux de complétion	30
5.2.1	2.1 Développement terminé	30
5.2.2	2.2 Tests réalisés	31
5.2.3	2.3 Revue de code	31
5.2.4	2.4 Intégration dans le repository	31
5.2.5	2.5 Documentation mise à jour	31
5.2.6	2.6 Fonctionnalité testée sur l'environnement de développement	31
5.2.7	2.7 Issue mise à jour dans l'outil de gestion de projet	32
5.3	3. Critères spécifiques selon le type de tâche	32
5.3.1	3.1 Fonctionnalité Backend.....	32

5.3.2	3.2 Fonctionnalité Frontend	32
5.3.3	3.3 Traitement de données / IA	32
5.4	4. Validation finale	32
5.5	5. Évolution de la Definition of Done	33
6	POPolitics – Fonctionnalités	33
6.1	1. Objectif général	33
6.2	2. Portail collectif	33
6.2.1	2.1 Historique et suivi des votes	33
6.2.2	2.2 Analyse des grandes tendances	34
6.2.3	2.3 Synthèses automatiques des débats	34
6.2.4	2.4 Indicateurs de cohérence des représentants	34
6.2.5	2.5 Visualisations interactives	34
6.3	3. Page personnelle	35
6.3.1	3.1 Notifications personnalisées	35
6.3.2	3.2 Vue simplifiée des actions et décisions des élus suivis	35
6.3.3	3.3 Fiches éclair express sur les lois, projets et amendements	35
7	POPolitics – Comparaison des stacks techniques	36
7.1	1. ETL & Orchestration	36
7.1.1	1.1 Kestra vs Airflow.....	36
7.2	2. Backend.....	36
7.2.1	2.1 Django vs FastAPI.....	36
7.2.2	2.2 Backend Python vs Backend JavaScript	37
7.3	3. Frontend	38
7.3.1	3.1 Next.js vs SPA React classique.....	38
7.4	4. IA & NLP	38
7.4.1	4.1 Framework de modélisation NLP	38
7.4.2	4.2 Exposition du service IA	39
7.5	5. Synthèse	39
8	POPolitics – Ressources matérielles & coûts.....	40
8.1	1. MVP local – Docker sur machine de développement.....	40
8.2	2. V1 – Serveur local (on-premise léger).....	40
8.3	3. Version Cloud – Hébergement managé.....	41
8.4	4. Cohérence avec les choix techniques	41

9	POPolitics – User Stories.....	42
9.1	1. Introduction	42
9.2	2. Personas	42
9.3	3. Épics.....	42
9.4	Epic 1 – Authentification & Gestion de compte.....	43
9.4.1	US-101 – Créer un compte	43
9.4.2	US-102 – Se connecter	43
9.4.3	US-103 – Se déconnecter	43
9.4.4	US-104 – Réinitialiser son mot de passe	44
9.5	Epic 2 – Portail collectif : Votes	44
9.5.1	US-201 – Consulter le tableau des votes	44
9.5.2	US-202 – Filtrer les votes	44
9.5.3	US-203 – Filtrer les votes par thème, commission et type de texte.....	45
9.5.4	US-204 – Consulter les indicateurs d’un vote.....	45
9.5.5	US-205 – Exporter les résultats de vote	45
9.6	Epic 3 – Portail collectif : Tendances & Analyses	46
9.6.1	US-301 – Visualiser les indicateurs de cohérence d’un élu	46
9.6.2	US-302 – Comparer deux élus ou deux groupes	46
9.6.3	US-303 – Visualiser la cartographie des alliances	46
9.6.4	US-304 – Croiser les données AN / Sénat / UE.....	46
9.7	Epic 4 – Portail collectif : Synthèses IA	47
9.7.1	US-401 – Obtenir un résumé automatique d’un débat	47
9.7.2	US-402 – Consulter une fiche éclair sur un texte de loi	47
9.8	Epic 5 – Page personnelle : Suivi & Notifications	48
9.8.1	US-501 – Suivre des élus	48
9.8.2	US-502 – Consulter la timeline d’un élu suivi	48
9.8.3	US-503 – Configurer ses thèmes d’intérêt.....	48
9.8.4	US-504 – Recevoir des notifications personnalisées	49
9.9	Epic 6 – Plateforme Data & ETL	49
9.9.1	US-601 – Ingérer les données de l’Assemblée nationale	49
9.9.2	US-602 – Ingérer les données du Sénat	49
9.9.3	US-603 – Ingérer les données du Parlement européen.....	50
9.9.4	US-604 – Normaliser les données en zone Silver	50

9.9.5	US-605 – Préparer les données analytiques en zone Gold	50
9.10	4. Récapitulatif	51
10	POPolitics - Plan de Sprints	51
10.1	1. Introduction	51
10.1.1	Principes.....	52
10.2	2. Vue d'ensemble.....	52
10.3	3. Detail des sprints	52
10.3.1	Sprint 1 - Fondations (Must).....	52
10.3.2	Sprint 2 - MVP Portail collectif.....	53
10.3.3	Sprint 3 - Enrichissements (Should)	53
10.3.4	Sprint 4 - IA et finitions (Should/Could)	54
10.4	4. Sprint 5 - Stabilisation & soutenance	54
10.5	5. Recapitulatif.....	54
11	POPolitics – Registre des risques	55
11.1	1. Méthode d'évaluation	55
11.2	2. Registre des risques	55
11.2.1	Catégorie : Données & Sources externes	55
11.2.2	Catégorie : Technique & Architecture	58
11.2.3	Catégorie : Organisation & Équipe	61
11.2.4	Catégorie : Périmètre & Livraison	63
11.3	3. Récapitulatif par criticité	64
11.3.1	Critiques (score 6–9).....	64
11.3.2	Moyens (score 3–4).....	64
11.3.3	Faibles (score 1–2).....	64
11.4	4. Suivi.....	65
12	POPolitics – Exigences non-fonctionnelles (NFR)	65
12.1	1. Introduction	65
12.2	2. Performance	65
12.2.1	2.1 Temps de réponse API	65
12.2.2	2.2 Pipeline ETL	65
12.2.3	2.3 Chargement frontend	66
12.3	3. Scalabilité	66
12.3.1	3.1 Charge utilisateurs	66

12.3.2	3.2 Volume de données.....	66
12.4	4. Disponibilité & fiabilité	66
12.4.1	4.1 Disponibilité cible	66
12.4.2	4.2 Gestion des pannes.....	67
12.4.3	4.3 Reprise après incident.....	67
12.5	5. Sécurité	67
12.5.1	5.1 Authentification & autorisation.....	67
12.5.2	5.2 Protection des données.....	67
12.5.3	5.3 Sécurité des échanges	67
12.5.4	5.4 Données personnelles (RGPD)	68
12.6	6. Maintenabilité	68
12.6.1	6.1 Lisibilité du code	68
12.6.2	6.2 Traçabilité	68
12.6.3	6.3 Évolutivité de l'architecture	68
12.7	7. Compatibilité.....	68
12.7.1	7.1 Navigateurs supportés	68
12.7.2	7.2 Résolutions	69
12.8	8. Contraintes d'infrastructure (MVP).....	69
13	POPolitics – Sources de données	69
13.1	1. Vue d'ensemble.....	69
13.2	2. Assemblée nationale.....	69
13.2.1	2.1 Portail	69
13.2.2	2.2 Données disponibles.....	69
13.2.3	2.3 Fréquence de mise à jour.....	70
13.2.4	2.4 Particularités	70
13.3	3. Sénat	70
13.3.1	3.1 Portail	70
13.3.2	3.2 Données disponibles.....	70
13.3.3	3.3 Fréquence de mise à jour.....	71
13.3.4	3.4 Particularités	71
13.4	4. Parlement européen	71
13.4.1	4.1 Portail	71
13.4.2	4.2 Données disponibles.....	71

13.4.3	4.3 Fréquence de mise à jour.....	71
13.4.4	4.4 Particularités	72
13.5	5. Formats et ingestion	72
13.5.1	5.1 Formats bruts attendus	72
13.5.2	5.2 Stratégie d’ingestion (pipeline ETL)	72
13.5.3	5.3 Identifiants croisés.....	72
13.6	6. Licences et conditions d’utilisation	73
13.7	7. Limitations connues.....	73
13.8	8. Évolutions envisagées	74
14	POPolitics – Sécurité & RGPD	74
14.1	1. Introduction	74
14.2	2. Périmètre des données	74
14.3	3. Authentification & gestion des accès.....	74
14.3.1	3.1 Mécanisme d’authentification.....	74
14.3.2	3.2 Stockage des mots de passe.....	74
14.3.3	3.3 Gestion des rôles	75
14.4	4. Sécurité des échanges	75
14.4.1	4.1 Transport.....	75
14.4.2	4.2 Protection des APIs	75
14.4.3	4.3 Protection contre les injections.....	75
14.5	5. Protection des secrets	76
14.5.1	5.1 Règles	76
14.5.2	5.2 Gestion en production	76
14.6	6. Obligations RGPD	76
14.6.1	6.1 Base légale du traitement	76
14.6.2	6.2 Droits des utilisateurs	76
14.6.3	6.3 Données collectées sur les utilisateurs	77
14.6.4	6.4 Minimisation des données	77
14.6.5	6.5 Consentement.....	77
14.7	7. Sécurité de l’infrastructure	77
14.7.1	7.1 Isolation des bases de données	77
14.7.2	7.2 Journalisation	77
14.7.3	7.3 Mises à jour de sécurité	78

14.8	8. Responsabilités	78
15	POPolitics – Stratégie de tests	78
15.1	1. Introduction	78
15.2	2. Principes généraux.....	78
15.3	3. Pyramide de tests	79
15.4	4. Types de tests par pôle.....	79
15.4.1	4.1 Backend (Django)	79
15.4.2	4.2 Pipeline ETL (Kestra / Python)	80
15.4.3	4.3 Frontend (Next.js)	80
15.5	5. Environnements de test.....	81
15.6	6. Intégration continue (CI)	81
15.7	7. Données de test.....	82
15.8	8. Responsabilités	82
15.9	9. Critères de qualité minimaux (MVP).....	82
16	POPolitics – Pipeline CI/CD	83
16.1	1. Introduction	83
16.2	2. Vue d’ensemble.....	83
16.3	3. Dépôts Git	83
16.4	4. Branches.....	84
16.5	5. Pipeline CI (Intégration continue)	84
16.5.1	5.1 Dépôt Documentation	84
16.5.2	5.2 Dépôt Backend (Django)	84
16.5.3	5.3 Dépôt Frontend (Next.js).....	85
16.5.4	5.4 Dépôt Data (ETL)	85
16.6	6. Règles de merge	85
16.7	7. Pipeline CD (Déploiement continu)	85
16.7.1	7.1 Environnements.....	85
16.7.2	7.2 Déploiement en démo	86
16.7.3	7.3 Services déployés	86
16.8	8. Variables d’environnement.....	86
16.9	9. Responsabilités	87
17	POPolitics – Modèle de données	87
17.1	1. Introduction	87

17.2	2. Vue d'ensemble des bases de données	87
17.3	3. Datamart Assemblée nationale.....	88
17.3.1	Schéma	88
17.3.2	Tables.....	88
17.4	4. Datamart Sénat	89
17.4.1	Tables principales	90
17.5	5. Datamart Parlement européen.....	91
17.5.1	Tables principales	91
17.6	6. Datamart Cross-match	92
17.6.1	Tables principales	92
17.7	7. Base Auth.....	92
17.7.1	Schéma	93
17.7.2	Tables.....	93
17.8	8. Index et performances	94
18	POPolitics - Diagramme de Gantt.....	94
18.1	1) Vue macro.....	95
18.2	2) Vue detaillee (toutes les US)	95
19	POPolitics – Solutions IA & Model Training	96
19.1	1. Vue d'ensemble.....	96
20	2. Architecture IA	97
20.1	2.1 Vue logique.....	97
20.2	2.2 Intégration applicative.....	97
21	3. Cas d'usage IA	97
21.1	3.1 Résumés automatiques (MVP)	97
21.1.1	Objectif.....	97
21.1.2	Sources	97
21.1.3	Pipeline	98
21.2	3.2 Score de similarité textes (MVP)	98
21.2.1	Objectif.....	98
21.2.2	Approche	98
21.2.3	Technologie	98
21.2.4	Exemple de sortie	98
21.3	3.3 Chatbot parlementaire (V1).....	98

21.3.1	Objectif.....	98
21.3.2	Architecture	98
21.3.3	Sources interrogées.....	98
21.3.4	Capacités	99
22	4. Modèles IA.....	99
22.1	4.1 Modèles LLM	99
22.2	4.2 Modèles d’embeddings	99
23	5. Stockage IA.....	99
23.1	5.1 Base vectorielle	99
24	6. Déploiement.....	99
24.1	6.1 Environnement	99
24.1.1	Développement :	99
24.1.2	Production :	99
25	7. Limites connues.....	100
26	Benchmarks POPolitics.....	100
26.1	0. Démarrage rapide bench . ps1	100
26.2	1. Backend Django vs FastAPI (overhead du framework).....	100
26.2.1	Prérequis	101
26.2.2	Lancer (automatique) — recommandé.....	101
26.2.3	Lancer (manuel, 2 terminaux).....	101
26.2.4	Mesurer les services réels	101
26.3	2. Frontend Next.js (SSR/SSG) vs SPA React	101
26.3.1	Prérequis	102
26.3.2	a) Core Web Vitals (Lighthouse).....	102
26.3.3	b) Crawlabilité / SEO (HTML brut).....	102
26.4	3. Où reporter les résultats.....	102

1 POPolitics – Personas

1.1 Introduction

Ce document décrit les **trois personas** du projet POPolitics. Chaque persona est une représentation fictive mais réaliste d’un type d’utilisateur cible. Ils servent de référence

commune pour les décisions de design, de priorisation des fonctionnalités et de rédaction des user stories.

Ces personas sont référencés dans [09-user-stories.md](#).

1.2 Persona 1 — Le Citoyen engagé

1.2.1 Profil

Nom	Marie Fontaine
Âge	34 ans
Profession	Infirmière en CHU
Localisation	Bordeaux (33)
Situation	En couple, deux enfants
Niveau numérique	Intermédiaire — utilise smartphone, réseaux sociaux, quelques apps métier

1.2.2 Biographie

Marie travaille en service de pédiatrie depuis 10 ans. Entre ses gardes, elle suit l'actualité politique sur son téléphone — principalement via les médias sociaux et des podcasts. Elle vote à chaque élection mais se sent souvent perdue face à la complexité des textes de loi et des positions politiques. Elle a développé une méfiance vis-à-vis des médias traditionnels qu'elle trouve trop orientés, et cherche à former ses propres opinions à partir de **données brutes et vérifiables**.

Elle entend souvent parler d'un élu en particulier sur les sujets de santé et d'hôpital public, mais n'a jamais trouvé un outil simple pour suivre concrètement ce que cet élu vote réellement au parlement.

1.2.3 Objectifs

- Comprendre comment **ses représentants locaux** votent sur les sujets qui la touchent (santé, éducation, logement).
- Pouvoir **vérifier rapidement** une information politique sans lire des dizaines de pages.
- Suivre l'**actualité parlementaire** en moins de 10 minutes par jour.
- Former son opinion sur la base de **faits, pas de discours**.

1.2.4 Frustrations

- Les sites officiels (Assemblée nationale, Sénat) sont **complexes et peu intuitifs**.
- Les médias simplifient trop ou au contraire noient l'information dans du contexte.

- Elle ne sait pas toujours **qui représente sa circonscription** ni ce qu’il a voté récemment.
- Les alertes d’actualité sont soit trop nombreuses, soit trop vagues.

1.2.5 Comportement numérique

Habitude	Détail
Appareils	Smartphone (70 %), ordinateur (25 %), tablette occasionnellement
Réseaux	Instagram, X (Twitter), podcasts
Sessions	Courtes (5–10 min), souvent en transport ou pendant les pauses
Tolérance aux frictions	Faible — abandonne si une page est trop lente ou confuse

1.2.6 Scénarios d’usage sur POPolitics

1. **Vérification rapide** : Elle entend à la radio qu’un député a voté contre une loi sur les infirmières. Elle ouvre POPolitics, tape le nom du député et vérifie le vote en 30 secondes.
2. **Suivi de son élu local** : Elle ajoute son député de circonscription à ses élus suivis et reçoit une notification quand il vote sur un texte lié à la santé.
3. **Découverte** : Via le portail collectif, elle découvre qu’un groupe politique qu’elle pensait proche de ses valeurs vote régulièrement contre les textes sur le service public hospitalier.

1.2.7 Citation représentative

« Je veux juste savoir ce que font réellement les gens que j’ai élus. Pas ce qu’ils disent à la télé — ce qu’ils votent vraiment. »

1.3 Persona 2 — Le Journaliste / Expert politique

1.3.1 Profil

Nom	Thomas Marchand
Âge	41 ans
Profession	Journaliste politique indépendant & blogueur data
Localisation	Paris (75)
Situation	Célibataire, très mobile
Niveau numérique	Intermédiaire-avancé — à l’aise avec Excel, Google Sheets, outils de visualisation en

1.3.2 Biographie

Thomas a travaillé 12 ans en presse nationale avant de se mettre à son compte. Il tient un blog d'analyse politique très suivi (~40 000 abonnés) et collabore ponctuellement avec des médias en ligne pour des articles de fond. Sa spécialité : le **data journalisme politique** — il transforme des données brutes en récits visuels compréhensibles pour le grand public.

Il passe plusieurs heures par semaine à extraire manuellement des données du site de l'Assemblée nationale et du Sénat, à les nettoyer dans des tableurs et à construire ses propres analyses. Ce travail fastidieux lui prend un temps précieux qu'il préférerait consacrer à l'écriture et à l'analyse.

1.3.3 Objectifs

- Accéder à des **données propres, structurées et à jour** sans effort d'extraction manuel.
- Pouvoir **croiser les données** entre AN, Sénat et Parlement européen sur un même élu ou groupe.
- Identifier rapidement des **tendances et anomalies** (votes atypiques, ruptures, alliances inattendues).
- **Exporter les données** pour les réutiliser dans ses articles et visualisations.
- Comparer des élus ou groupes sur une période longue.

1.3.4 Frustrations

- Les sites officiels (AN, Sénat, UE) sont **disparates et peu lisibles** : il faut naviguer entre plusieurs interfaces pour trouver les mêmes infos.
- Le croisement manuel entre AN et Sénat sur un même élu est **extrêmement chronophage**.
- Les outils existants sont soit trop généralistes, soit derrière des paywalls prohibitifs.
- Les données brutes disponibles sont dans des formats difficiles à exploiter sans bagage technique.

1.3.5 Comportement numérique

Habitude	Détail
Appareils	Ordinateur (Mac, 95 %)
Outils	Google Sheets, Datawrapper, Flourish, Canva
Sessions	Longues (1–3h), le matin ou tard le soir
Tolérance aux frictions	Haute — accepte la complexité si les données sont fiables et riches

1.3.6 Scénarios d’usage sur POPolitics

1. **Enquête thématique** : Il prépare un article sur les votes concernant le nucléaire depuis 2015. Il filtre par thème, période et institution, exporte le CSV et l’importe dans Datawrapper pour créer une visualisation.
2. **Portrait d’élus** : Il accède à la fiche d’un sénateur, consulte son score de cohérence avec son groupe, identifie 3 votes atypiques et les utilise comme angle d’attaque pour son article.
3. **Analyse des alliances** : Il utilise la cartographie des alliances pour identifier quels groupes ont voté ensemble sur les textes économiques lors de la dernière session. Il inclut la capture dans son article.
4. **Croisement AN/Sénat/UE** : Pour un article sur un eurodéputé qui a également été député AN, il croise ses votes sur les trois institutions pour montrer une évolution de positionnement au fil du temps.

1.3.7 Citation représentative

« Ce dont j’ai besoin, c’est d’un accès propre aux données — pas d’une interface grand public. Donnez-moi les filtres, les exports et les comparateurs, et je fais le reste. »

1.4 Persona 3 — L’Administrateur plateforme

1.4.1 Profil

Nom	Lucas Perrin
Âge	26 ans
Profession	Étudiant M2 Informatique / Membre de l’équipe POPolitics
Localisation	Lyon (69)
Rôle dans l’équipe	DevOps / Data Engineer en alternance
Niveau numérique	Expert — DevOps, Docker, CI/CD, monitoring, Python

1.4.2 Biographie

Lucas est en dernière année de master à Epitech. Il a rejoint le projet POPolitics dès le départ, convaincu par la dimension civic-tech du projet. En entreprise, il travaille chez un éditeur SaaS B2B où il gère l’infrastructure cloud et les pipelines de données. Il apporte à l’équipe ses compétences en orchestration, déploiement et fiabilité des systèmes.

Au quotidien, il est l’un des seuls membres de l’équipe à interagir directement avec l’infrastructure de production : il surveille les pipelines Kestra, diagnostique les erreurs

d'ingestion, gère les accès et s'assure que la plateforme est disponible avant chaque démo ou soutenance.

1.4.3 Objectifs

- **Maintenir les pipelines ETL** opérationnels sans intervention manuelle quotidienne.
- Détecter et corriger rapidement les **erreurs d'ingestion** (API source indisponible, format modifié, rejet de données).
- Gérer les **comptes utilisateurs** et les droits d'accès à la plateforme.
- Préparer et valider l'environnement avant les **démos et soutenances**.
- Monitorer la **santé globale** du système (services, base de données, jobs Kestra).

1.4.4 Frustrations

- Les erreurs silencieuses dans les pipelines : un job ETL échoue mais aucune alerte n'est déclenchée, les données en base sont obsolètes sans que personne ne le remarque.
- La **dépendance aux APIs publiques** instables : un changement de format sur data.assemblee-nationale.fr peut casser toute la chaîne d'ingestion.
- La gestion manuelle des **migrations de base de données** lors des mises à jour.
- Le manque de visibilité sur l'état global de la plateforme en dehors de la console Kestra.

1.4.5 Comportement numérique

Habitude	Détail
Appareils	Ordinateur (Linux/Mac), terminal omniprésent
Outils	Docker, GitHub Actions, Kestra UI, psql, logs système
Sessions	Ponctuelles et réactives — intervient sur incident ou lors des déploiements
Tolérance aux frictions	Très haute pour les outils techniques, très faible pour le manque de logs clairs

1.4.6 Scénarios d'usage sur POPolitics

1. **Monitoring quotidien** : Chaque matin, il consulte le tableau de bord Kestra pour vérifier que les jobs ETL de la nuit se sont bien exécutés. Si un job est en erreur, il accède aux logs, identifie la cause (timeout API, schéma modifié) et relance manuellement ou déclenche un correctif.
2. **Gestion des utilisateurs** : Un membre de l'équipe pédagogique demande un accès démo à la plateforme. Lucas crée un compte via l'interface d'administration, lui attribue le rôle "lecture seule" et lui communique ses identifiants.

3. **Déploiement avant soutenance** : 48h avant une soutenance, il exécute le script de déploiement complet, vérifie les migrations, charge un jeu de données de démo figé et effectue les smoke tests pour s'assurer que tous les parcours utilisateurs critiques fonctionnent.
4. **Gestion d'incident** : Un pipeline d'ingestion des données du Sénat échoue deux jours de suite. Lucas analyse les logs, identifie que l'URL de l'API source a changé, met à jour le connecteur Kestra et relance le pipeline avec rejeu des données manquantes.

1.4.7 Citation représentative

« Mon job, c'est que personne ne remarque que j'existe — parce que si la plateforme tourne sans accroc, c'est que j'ai bien fait mon travail. »

1.5 Récapitulatif

Persona	Représente	Priorité fonctionnelle
Marie Fontaine — Citoyenne	Grand public, usage mobile, sessions courtes	Simplicité, notifications, filtres de base
Thomas Marchand — Journaliste/Expert	Utilisateur avancé, analyse data, desktop	Export, filtres avancés, cross-match, comparateurs
Lucas Perrin — Administrateur	Équipe interne, opérations, infrastructure	Monitoring, gestion utilisateurs, fiabilité ETL

2 POPolitics – Choix techniques

2.1 1. Vision générale

- Projet de **collecte, transformation, analyse** et **vulgarisation** de données politiques publiques.
 - **Sources** : Assemblée nationale, Sénat, Parlement européen.
 - **Contraintes** :
 - Budget nul au départ
 - Données publiques
 - Architecture évolutive
 - **Message clé** : Construire un pipeline **moderne, robuste** et **scalable, gratuit au départ**.
-

2.2 2. Objectifs fonctionnels de l'application

2.2.1 2.1 Objectifs principaux

- Centraliser les données politiques issues de plusieurs institutions.
- Nettoyer, structurer et croiser les données.
- Rendre l'information accessible et compréhensible pour un public non expert.
- Offrir une interface web performante et sécurisée.

2.2.2 2.2 Contraintes clés

- Sources multiples (AN, Sénat, UE).
 - Volumétrie évolutive.
 - Besoin de traçabilité et de reproductibilité.
 - Séparation claire des responsabilités (data / backend / frontend).
-

2.3 3. Vision globale de l'architecture

2.3.1 3.1 Approche retenue

- Architecture **modulaire en couches**.
- Séparation claire entre :
 - Pipeline ETL
 - Stockage analytique
 - Services backend
 - Interface frontend

2.3.2 3.2 Motivations

- Scalabilité (MVP local → évolution Cloud, ajout de sources, etc.).
 - Maintenabilité.
 - Possibilité d'évolution indépendante de chaque brique.
 - Alignement avec des architectures utilisées en production.
-

2.4 4. Sources de données & ingestion

2.4.1 4.1 Sources exploitées

- Données du **Sénat**.
- Données de l'**Assemblée nationale**.
- Données du **Parlement européen**.

2.4.2 4.2 Caractéristiques

- Formats hétérogènes.
- Fréquences de mise à jour variables.
- Structures parfois non normalisées.

2.4.3 4.3 Choix

- Phase **EXTRACT** dédiée.
 - Centralisation initiale avant toute transformation.
-

2.5 5. Landing Zone & Data Lake

2.5.1 5.1 Landing Zone – Data Lake

- Stockage de type fichiers/objets, pensé pour être **scalable**, qu'il soit hébergé en local (MVP, V1 serveur) ou dans le **Cloud**.
- Format **Parquet** / fichiers bruts.

2.5.2 5.2 Organisation en 3 niveaux

- **Bronze** : données brutes (traçabilité maximale).
- **Silver** : données nettoyées et pré-traitées.
- **Gold** : données prêtes à être consommées.

2.5.3 5.3 Avantages

- Séparation des responsabilités.
 - Reproductibilité des traitements.
 - Facilité de debug et d'audit.
-

2.6 6. Orchestration des pipelines ETL

2.6.1 6.1 Outil : Kestra

Rôle :

- Orchestration des tâches ETL.
- Gestion des dépendances.
- Automatisation des workflows.

2.6.2 6.2 Pourquoi Kestra ?

- Orienté **data engineering**.
- Déclaratif et lisible (**YAML**).
- Facilement observable (logs, retries, monitoring).

- Outil **français**.
-

2.7 7. Stockage analytique : Datamarts

2.7.1 7.1 Outil : PostgreSQL

Organisation :

- Datamart 1 : Assemblée nationale.
- Datamart 2 : Sénat.
- Datamart 3 : Parlement européen.
- Datamart 4 : Cross-match multi-institutions.

2.7.2 7.2 Raisons du choix

- SQL standard et robuste.
 - Adapté à l'analyse tabulaire.
 - Bonne intégration avec les services backend.
-

2.8 8. Couche IA & Model Training

2.8.1 8.1 Socle IA

- Données issues de la couche **Silver/Gold**.

2.8.2 8.2 Cas d'usage

- Résumés automatiques des lois et amendements votés (MVP)
- Score de similarité lois / amendements et déclaration de chaque groupe (MVP)
- Chatbot (V1)

2.8.3 8.3 Intégration

- Les modèles IA sont exécutés dans un service dédié (IA Service), indépendant du frontend.
 - Le backend Django consomme ce service via API pour les fonctionnalités IA (chatbot, résumé, similarité, classification).
 - Le service IA orchestre les appels LLM, la recherche vectorielle (RAG) et les requêtes SQL.
 - Déploiement possible avec API LLM externes.
-

2.9 9. Backend – Django

2.9.1 9.1 Découpage en services

- **IA Service** : accès aux modèles et prédictions.
- **Data Service** : exposition des données métier issues des datamarts.
- **Auth Service** : gestion des utilisateurs et des droits.

2.9.2 9.2 Communication

- APIs **REST**.
- Découplage fort entre services backend et plate-forme data.

2.9.3 9.3 Objectifs

- Clarté fonctionnelle.
 - Sécurité.
 - Évolutivité indépendante.
-

2.10 10. Authentification & gestion des utilisateurs

2.10.1 10.1 Base d'authentification dédiée

- Stockage des utilisateurs.
- Rôles et permissions.

2.10.2 10.2 Flux

- Frontend → Auth Service → Auth Database.

2.10.3 10.3 Objectifs

- Sécurité.
 - Séparation des données métier et utilisateurs.
 - Préparation à des fonctionnalités avancées (comptes, profils, favoris).
-

2.11 11. Frontend – Next.js

2.11.1 11.1 Architecture

- **Next Server** : BFF (Backend For Frontend).
- **Server Actions** : logique côté serveur.
- **Next Client** : rendu côté utilisateur.

2.11.2 11.2 Motivations

- Performance (**SSR / SSG**).

- Sécurité (logique sensible côté serveur).
 - Intégration naturelle avec des APIs REST.
-

2.12 12. Synthèse

- Séparation claire des responsabilités (ETL, stockage, services, UI).
- Application des bonnes pratiques du **data engineering moderne**.
- Structuration des données via **Data Lake** (Bronze / Silver / Gold).
- Datamarts dédiés aux usages analytiques.
- Exposition des données via **APIs REST**.
- BFF garantissant sécurité et cohérence côté frontend.

L'architecture est cohérente avec les objectifs fonctionnels, réaliste pour un projet de Master 2 et alignée avec les standards professionnels, tout en restant généralisable à d'autres contextes institutionnels.

3 POPolitics – Priorisation MoSCoW

3.1 Méthode

La priorisation MoSCoW est utilisée ici pour distinguer :

- **Must have** : indispensable pour un MVP crédible à la soutenance.
 - **Should have** : important, fortement souhaité si le temps le permet.
 - **Could have** : bonus / nice-to-have si la charge le permet.
 - **Won't have (for now)** : hors périmètre de ce projet (phase ultérieure).
-

3.2 1. Must have

3.2.1 1.1 Données & ETL (plateforme minimale)

- Ingestion des données **AN, Sénat et UE** via un pipeline ETL minimal.
- Mise en place de la Landing Zone avec au moins les couches **Bronze** (brut) et **Silver** (nettoyé / normalisé).
- Orchestration de base des jobs ETL avec **Kestra** (lancement, logs, statut des exécutions).
- Création de datamarts PostgreSQL pour au moins **AN, Sénat et UE**, consommables par le backend.

3.2.2 1.2 Backend & services

- Backend **Django** opérationnel, structuré en services :

- **Data Service** exposant des APIs REST pour le **Portail collectif** (votes, indicateurs de base),
- **Auth Service** avec authentification basique (création de compte, login, rôles simples).
- Intégration minimale avec la plate-forme data (requêtes sur les datamarts principaux).

3.2.3 1.3 Frontend (MVP Portail collectif)

- Frontend **Next.js** avec BFF (Next Server) consommant les APIs backend.
- Pages principales :
 - Page de **connexion / inscription**,
 - Première version du **Portail collectif** :
 - tableau des votes avec filtres essentiels (institution, période, groupe politique, élu),
 - premiers graphiques simples (évolution, répartition des votes).
- Gestion minimale des erreurs et états de chargement.

3.2.4 1.4 Qualité, sécurité & documentation

- Traçabilité minimale des traitements ETL (logs Kestra, identifiants de batch, statut des jobs).
- Séparation claire des données métier et des données d'authentification (base Auth dédiée).
- Livraison d'une **documentation d'architecture** (choix techniques, schémas) et d'un **Plan Qualité** décrivant methodo, tests et déploiement.

3.3 2. Should have

3.3.1 2.1 Données & analytique

- Mise en place de la couche **Gold** dans le Data Lake (données prêtes à l'analyse).
- Datamart **Cross-match** permettant de croiser AN / Sénat / UE.
- Jeux d'**indicateurs et KPI** plus avancés (activité, typologie de votes, cohérence, opposition, alliances).

3.3.2 2.2 Couche IA

- Premier modèle IA en production (ex. **résumé automatique** de débats ou **classification** de textes politiques).
- **IA Service** exposant au moins un endpoint d'inférence vers le backend.

3.3.3 2.3 Frontend (tendances & page personnelle v1)

- Filtres avancés (dates, institutions, catégories, types de texte).

- Visualisations graphiques plus évoluées (courbes, barres, heatmaps simples).
- Première version de la **Page personnelle** :
 - sélection d'élus suivis et de thèmes d'intérêt,
 - timeline simplifiée des actions des élus suivis,
 - premières **notifications personnalisées** basées sur ces préférences.

3.3.4 2.4 Opérations & outillage

- Monitoring plus détaillé des workflows Kestra (retries, alertes simples, tableau de bord minimal).
- Scripts de déploiement / configuration plus industrialisés (Docker ou équivalent) pour la stack Django / Next.js / Kestra.

3.4 3. Could have

3.4.1 3.1 Données & IA

- Modèles IA supplémentaires (analyse de tendance plus fine, détection d'anomalies, recommandation de contenus, etc.).
- Scénarios de simulation ou de projection simples à partir des données.

3.4.2 3.2 Frontend & UX

- Tableau de bord **hautement personnalisable** par utilisateur (widgets, réorganisation des blocs).
- Export avancé des résultats (CSV, PDF, infographies prêtes pour les réseaux sociaux ou la presse).
- Pages explicatives / pédagogiques détaillées sur les données, les indicateurs et la méthodologie.

3.4.3 3.3 Plateforme & DevOps

- Pipeline CI/CD automatisé (tests, build, déploiement automatique sur un environnement de démo).
- Supervision centralisée (tableau de bord de santé des services et des jobs Kestra, alertes proactives).

3.5 4. Won't have (for now)

- Ingestion **temps réel** ou streaming massif de données.
- Infrastructure cloud complexe et coûteuse (cluster managé, data warehouse propriétaire payant, etc.).
- Application mobile native dédiée (Android / iOS).
- Fonctionnalités sociales avancées (commentaires, interactions entre utilisateurs).

- Mécanismes de recommandation complexes basés sur l'IA.

Ces éléments sont considérés hors périmètre pour le projet et pourront être envisagés dans une phase ultérieure si le projet est pérennisé.

4 POPolitics – Exigences fonctionnelles et non fonctionnelles liées au WBS

Ce document complète le **WBS** en décrivant, pour chaque grand lot, les **exigences fonctionnelles** (ce que le système doit faire) et **non fonctionnelles** (qualité, performances, sécurité, UX, etc.).

4.1 1. Portail collectif (WBS – Fonctionnalité principale)

4.1.1 1.1 Description

Interface principale permettant de **suivre les votes, tendances et débats** à l'échelle collective (groupes politiques, institutions, textes de loi).

4.1.2 1.2 Exigences fonctionnelles

- Afficher l'historique des votes avec filtres par :
 - institution (AN, Sénat, UE),
 - période,
 - type de texte (loi, amendement, résolution...),
 - groupe politique, élu, commission.
- Afficher les résultats de vote avec codes couleurs (Pour / Contre / Abstention / Non-votant).
- Calculer et afficher :
 - taux de participation,
 - cohésion de groupe,
 - évolution temporelle.
- Fournir des visualisations de base (tableaux + graphiques).

4.1.3 1.3 Exigences non fonctionnelles

- Temps de réponse acceptable pour les principales requêtes (ex. < 2–3 secondes sur un dataset de taille raisonnable).
- Interface responsive (desktop en priorité, utilisation possible sur tablette).
- Lisibilité : couleurs cohérentes, légendes et titres explicites.

4.1.4 1.4 Critères d'acceptation (exemples)

- Un utilisateur peut, en moins de 5 clics, afficher les votes d'un groupe politique sur une période donnée.
 - Les indicateurs de cohésion et de participation sont systématiquement mis à jour quand les filtres changent.
-

4.2 2. Page personnelle & notifications (WBS – Fonctionnalité utilisateur)

4.2.1 2.1 Description

Tableau de bord individuel permettant à un utilisateur de suivre **ses élus, ses thèmes** et de recevoir des **notifications personnalisées**.

4.2.2 2.2 Exigences fonctionnelles

- L'utilisateur peut :
 - sélectionner des élus suivis,
 - choisir des thèmes d'intérêt (économie, social, environnement...).
- Le système génère des notifications en fonction :
 - d'événements importants (vote majeur, changement de position, amendement marquant),
 - des préférences de l'utilisateur.
- Afficher une timeline des principales actions des élus suivis.

4.2.3 2.3 Exigences non fonctionnelles

- Volume de notifications limité pour éviter la fatigue (priorisation par importance / urgence / intérêt).
- Possibilité de désactiver certaines catégories d'alertes.

4.2.4 2.4 Critères d'acceptation

- Un utilisateur peut configurer ses préférences et ne reçoit ensuite que des notifications en lien avec celles-ci.
 - Les notifications affichent un lien direct vers la page correspondante (vote, texte, fiche élu).
-

4.3 3. Plateforme Data & ETL (WBS – Infrastructure data)

4.3.1 3.1 Description

Ensemble des pipelines ETL et du Data Lake (Landing Zone Bronze / Silver / Gold) permettant d'ingérer, nettoyer et structurer les données.

4.3.2 3.2 Exigences fonctionnelles

- Ingestion régulière des données AN, Sénat, UE.
- Stockage des données brutes en zone **Bronze** (traçabilité).
- Transformation vers **Silver** (nettoyage, normalisation).
- Préparation des données prêtes pour analyse en zone **Gold**.
- Orchestration des pipelines via **Kestra**.

4.3.3 3.3 Exigences non fonctionnelles

- Reproductibilité des traitements (mêmes entrées → mêmes sorties).
- Traçabilité des exécutions (logs, statut des jobs).
- Capacité à relancer un job en erreur sans casser les données.

4.3.4 3.4 Critères d'acceptation

- Pour une nouvelle source intégrée, le pipeline peut être relancé de bout en bout sans intervention manuelle lourde.
 - Les jeux de données Bronze, Silver et Gold sont clairement identifiables et documentés.
-

4.4 4. Datamarts & analytique (WBS – Stockage analytique)

4.4.1 4.1 Description

Datamarts PostgreSQL dédiés aux analyses politiques (AN, Sénat, UE, Cross-match).

4.4.2 4.2 Exigences fonctionnelles

- Mise à disposition de tables analytiques pour :
 - les votes par texte / élu / groupe,
 - les indicateurs agrégés,
 - les croisements entre institutions (datamart Cross-match).
- Actualisation des datamarts en cohérence avec les pipelines ETL.

4.4.3 4.3 Exigences non fonctionnelles

- Schémas normalisés et documentés.
- Requêtes principales exécutables en temps raisonnable (index adaptés).

4.4.4 4.4 Critères d'acceptation

- Les APIs backend peuvent récupérer les données nécessaires via des requêtes SQL stables et documentées.
-

4.5 5. Couche IA & modèles (WBS – IA Service)

4.5.1 5.1 Description

Socle IA fournissant des **résumés**, **classifications** ou autres analyses automatiques.

4.5.2 5.2 Exigences fonctionnelles

- Entraîner au moins un modèle (ex. résumé de débat ou classification de texte).
- Exposer un endpoint d'inférence via l'**IA Service**.

4.5.3 5.3 Exigences non fonctionnelles

- Temps de réponse compatible avec une utilisation interactive (ex. < 5 s pour un résumé court).
- Possibilité de journaliser les appels pour améliorer les modèles ultérieurement.

4.5.4 5.4 Critères d'acceptation

- Depuis le frontend, un utilisateur peut demander un résumé ou une analyse IA et obtenir une réponse lisible.
-

4.6 6. Authentification & gestion des utilisateurs (WBS – Auth Service)

4.6.1 6.1 Description

Service d'authentification et base d'utilisateurs dédiés.

4.6.2 6.2 Exigences fonctionnelles

- Création de compte, login / logout.
- Rôles simples (ex. utilisateur standard, admin).
- Stockage sécurisé des mots de passe.

4.6.3 6.3 Exigences non fonctionnelles

- Respect des bonnes pratiques de sécurité (hashage robuste, pas de mot de passe en clair).
- Séparation stricte entre données d'authentification et données métier.

4.6.4 6.4 Critères d'acceptation

- Impossible d'accéder aux pages protégées sans être authentifié.
 - Les droits d'accès sont appliqués conformément au rôle de l'utilisateur.
-

4.7 7. Qualité, monitoring & opérations (WBS – Support)

4.7.1 7.1 Description

Ensemble des pratiques de qualité, de tests et de supervision de la plateforme.

4.7.2 7.2 Exigences fonctionnelles

- Mise en place de tests (voir Plan Qualité).
- Supervision minimale des jobs Kestra et des services backend.

4.7.3 7.3 Exigences non fonctionnelles

- Logs accessibles et compréhensibles pour diagnostiquer un incident.
- Documentation à jour des procédures de redémarrage ou de réinstallation.

Ce document est volontairement synthétique et doit être lu conjointement avec : - le **WBS** (découpage du travail), - les documents de **fonctionnalités** et de **MoSCoW**, - le **Plan Qualité**.

5 POPolitics – Definition of Done (DoD)

5.1 1. Objectif

La **Definition of Done (DoD)** définit les critères permettant de considérer qu'une tâche, une fonctionnalité ou une user story est terminée et prête à être livrée.

Elle garantit que :

- toutes les fonctionnalités respectent un niveau de qualité défini ;
- le code est testé, documenté et validé ;
- l'équipe partage une définition commune du travail terminé.

La DoD permet également de maintenir une qualité constante du produit tout au long du projet.

5.2 2. Critères généraux de complétion

Une tâche est considérée comme **terminée** lorsque les conditions suivantes sont respectées.

5.2.1 2.1 Développement terminé

- Le code correspondant à la fonctionnalité est entièrement implémenté.
- Le code respecte les bonnes pratiques de développement définies par l'équipe.
- Le code est structuré et lisible.

5.2.2 2.2 Tests réalisés

Les tests nécessaires ont été réalisés :

- tests unitaires ;
- tests d'intégration (si applicable) ;
- tests manuels de validation.

Les critères suivants doivent être respectés :

- les tests passent **sans erreur** ;
- aucun bug critique n'est détecté.

5.2.3 2.3 Revue de code

Une **code review** a été réalisée par au moins un autre membre de l'équipe. La revue vérifie :

- la qualité du code ;
- la cohérence avec l'architecture ;
- le respect des standards de développement.

Les retours de relecture doivent être **corrigés avant validation**.

5.2.4 2.4 Intégration dans le repository

La fonctionnalité doit être :

- commit dans une branche dédiée ;
- fusionnée dans la branche principale via Pull Request ;
- validée après revue de code.

5.2.5 2.5 Documentation mise à jour

La documentation liée à la fonctionnalité doit être mise à jour si nécessaire :

- README ;
- documentation technique ;
- documentation API ;
- commentaires dans le code.

5.2.6 2.6 Fonctionnalité testée sur l'environnement de développement

La fonctionnalité doit être :

- testée dans l'application ;
- vérifiée pour s'assurer qu'elle fonctionne correctement ;
- validée par l'équipe.

5.2.7 2.7 Issue mise à jour dans l'outil de gestion de projet

Lorsque la tâche est terminée :

- l'issue correspondante est mise à jour ;
 - elle est déplacée dans la colonne **Done** du board de gestion de projet ;
 - les commentaires nécessaires sont ajoutés (lien vers la PR, notes importantes, etc.).
-

5.3 3. Critères spécifiques selon le type de tâche

5.3.1 3.1 Fonctionnalité Backend

Pour une fonctionnalité backend :

- l'API fonctionne correctement ;
- les endpoints répondent avec les bons formats et codes de retour ;
- les erreurs sont correctement gérées (messages clairs, statuts HTTP adaptés).

5.3.2 3.2 Fonctionnalité Frontend

Pour une fonctionnalité frontend :

- l'interface fonctionne sans erreur ;
- l'affichage est cohérent avec le design défini (UX/UI) ;
- les interactions utilisateur fonctionnent correctement sur les parcours ciblés.

5.3.3 3.3 Traitement de données / IA

Pour les fonctionnalités liées aux données ou à l'IA :

- les données sont correctement traitées (qualité, complétude minimale) ;
 - les résultats produits sont cohérents par rapport aux attentes fonctionnelles ;
 - les performances sont acceptables (temps de traitement raisonnable pour l'usage visé).
-

5.4 4. Validation finale

Une tâche est officiellement considérée comme **terminée** lorsque :

- tous les critères de la Definition of Done sont respectés ;
 - la Pull Request est validée ;
 - la fonctionnalité est intégrée au projet ;
 - l'issue correspondante est marquée comme **Done** dans l'outil de gestion.
-

5.5 5. Évolution de la Definition of Done

La Definition of Done peut évoluer au cours du projet afin de :

- améliorer la qualité du produit ;
- adapter les processus de développement ;
- intégrer les retours de l'équipe.

Toute modification doit être **discutée et validée collectivement** par l'équipe, puis répercutée dans le présent document.

6 POPolitics – Fonctionnalités

6.1 1. Objectif général

Objectif : rendre le suivi de l'activité parlementaire simple, clair et interactif.

Popolitics centralise les informations issues des institutions politiques et les transforme en insights actionnables pour :

- les citoyens,
- les journalistes,
- les experts politiques.

Valeur ajoutée :

- Données fiables et mises à jour quotidiennement.
- Analyses automatisées via des fonctionnalités IA.
- Expérience utilisateur soignée : interface et visualisations claires et personnalisées.
- Gain de temps : résumés rapides, notifications personnalisées.

Les fonctionnalités présentées restent à préciser, tout comme les technologies et frameworks utilisés : ils seront définis lorsque l'équipe sera au complet.

6.2 2. Portail collectif

Le cœur de la plateforme, pour analyser les votes, tendances et débats.

6.2.1 2.1 Historique et suivi des votes

« Qui vote quoi, en un clin d'œil ! »

- Mise à jour quotidienne automatisée.
- Filtres avancés : thème, période, commission, type de texte, amendement vs vote final.

- Filtres de catégories : groupe politique, élu, rôle (rapporteur, ministre...).
- Codes couleurs intuitifs (Pour / Contre / Abstention / Non-votant).
- Calculs dynamiques : taux de participation, cohésion du groupe, évolution temporelle.
- Export intelligent : PDF, CSV, infographies.

Mockup suggéré : tableau dynamique + graphique d'évolution des votes.

6.2.2 2.2 Analyse des grandes tendances

« Quelles dynamiques politiques en temps réel ? »

- Détection des sujets émergents (NLP + clustering).
- Indice d'opposition entre groupes.
- Cartographie des alliances (ex. LR + RN sur certains textes).
- Heatmaps d'alignement.
- Graphes de centralité politique : « qui influence réellement quoi ? ».

6.2.3 2.3 Synthèses automatiques des débats

« Un résumé flash pour rester connecté-e facilement à l'information politique »

- Résumé en 5 points clés.
- Mise en avant des arguments dominants.
- Détection des “mots forts” et du ton des interventions.
- Accès direct aux passages vidéo liés à chaque point.

Mockup suggéré : bouton pour accéder au débat complet et aux documents sources.

6.2.4 2.4 Indicateurs de cohérence des représentants

« Quel représentant se démarque ? Qui suit le groupe ? »

- Indice de stabilité des positions.
- Alignement au groupe politique (cohésion verticale).
- Analyse des exceptions : votes atypiques, ruptures, évolutions.
- Comparaison entre élus ou groupes.
- Détection des profils atypiques.

Mockup suggéré : jauges et comparateurs par élu et par groupe.

6.2.5 2.5 Visualisations interactives

« Comment rendre l'information politique ludique ? »

- Graphe de réseau politique (similarité des votes).
- Chronologie d'un texte (amendements → passage en commission → vote final).
- Clusters de positionnement idéologique (t-SNE, PCA...).

- Comparateur d'élus ou de groupes.
- Carte thermique des votes par thème.

Mockup suggéré : timeline + graphe de clusters avec zones de couleur.

6.3 3. Page personnelle

Un tableau de bord individuel pour rester informé·e rapidement, selon ses centres d'intérêts.

6.3.1 3.1 Notifications personnalisées

« Impliquer le/la citoyen·ne tout en respectant ses choix »

- Système de priorisation : importance + urgence + intérêt utilisateur.
- Alertes « texte majeur », changement de position d'un élu, amendement surprenant :
 - des alertes qui ne fatiguent pas, en prévenant uniquement quand cela compte vraiment.
- Ajustement dynamique : la plateforme apprend les centres d'intérêt de l'utilisateur.

Mockup suggéré : bannière de notification colorée avec icône de catégorie (économie, social, environnement...).

6.3.2 3.2 Vue simplifiée des actions et décisions des élus suivis

« Qu'est-ce qu'a fait ce représentant ? »

- Timeline des actions de chaque élu : interventions, amendements, rendez-vous publics.
- Profil express : bio, rôle, commissions.

Mockup suggéré : mini-carte avec photo et tendance des votes.

6.3.3 3.3 Fiches éclair express sur les lois, projets et amendements

« Comprendre l'essentiel des textes en moins d'une minute »

- Résumé immédiat : objectif, mécanisme, impacts.
- Analyse des enjeux clés.
- Qui est pour / qui est contre ? Pourquoi ?
- Schéma simplifié du processus législatif du texte.
- « Carte d'influence » : penseurs, lobbies, grandes organisations impliquées.

Mockup suggéré : flashcards interactives avec 3 vues : **Résumé / Enjeux / Acteurs**.

7 POPolitics – Comparaison des stacks techniques

Ce document présente, de manière synthétique, les principales **options techniques étudiées** pour POPolitics, séparées par couche (ETL, Backend, Frontend), et justifie les choix retenus.

7.1 1. ETL & Orchestration

7.1.1 1.1 Kestra vs Airflow

O
pt
io
n

	Avantages principaux	Limites / remarques	Choix retenu
Kestra	Orienté data engineering, workflows déclaratifs en YAML, interface moderne, déploiement simple en Docker, bonne observabilité, outil français	Communauté plus jeune qu’Airflow, moins de documentation historique	Oui – bon compromis pédagogie / simplicité / modernité
Airflow	Standard industriel très répandu, très puissant, riche en opérateurs	Plus lourd à mettre en place et à administrer pour un projet étudiant, courbe d’apprentissage plus importante	Non pour le MVP (envisageable plus tard)

Conclusion : Kestra est choisi pour **orchestrer les pipelines ETL** car il offre un bon équilibre entre puissance, lisibilité et simplicité de déploiement dans une stack Docker multi-conteneurs, tout en étant un **outil français**, cohérent éthiquement avec un projet centré sur la politique française et la souveraineté numérique. Ce choix complète l’architecture **Data Lake (Bronze / Silver / Gold)** décrite dans 01-technological-choices.md en apportant une orchestration moderne et facilement reproductible.

7.2 2. Backend

7.2.1 2.1 Django vs FastAPI

O
p
ti
o
n

	Avantages principaux	Limites / remarques	Choix retenu
--	----------------------	---------------------	--------------

O
p
ti
o
n

D
j
a
n
g
o

F
a
s
t
A
P
I

Option	Avantages principaux	Limites / remarques	Choix retenu
	Framework web complet (ORM, auth, admin), écosystème mature, très adapté aux CRUD et aux APIs REST classiques	Plus « lourd » que FastAPI pour de la simple API, courbe d'apprentissage un peu plus large	Oui – cohérent avec le besoin d'un backend complet (Auth Service, Data Service, IA Service)
	Très rapide, orienté API, typing moderne, excellent pour des micro-services légers	Nécessite d'assembler plus de briques (auth, admin, etc.), moins aligné avec le besoin d'un backend structuré « tout-en-un »	Non, réservé éventuellement à des services spécifiques ultérieurs

Conclusion : Django est retenu comme **socle backend principal** pour POPolitics, afin de bénéficier d'un cadre complet (authentification, ORM, structuration des apps) adapté au MVP et à la taille de l'équipe, et conforme à l'architecture **multi-services** (Data Service, IA Service, Auth Service) décrite dans 01-technological-choices.md.

7.2.2 2.2 Backend Python vs Backend JavaScript

Option	Avantages principaux	Limites / remarques	Choix retenu
Backend Python (Django)	Aligné avec l'écosystème data/IA (ETL en Python, socle IA exposé en FastAPI), forte cohérence technique, mutualisation des compétences	Moins homogène avec le frontend JavaScript, nécessité de maîtriser deux langages sur le projet	Oui – meilleure intégration avec ETL & IA, moins de friction technique globale
Backend JavaScript (Node.js, Express/Nest)	Homogénéité full-JS (front + back), grande communauté, écosystème riche de packages	Moins naturel pour interfacer directement les pipelines ETL Python et les modèles IA Python ; nécessite plus de "colle" entre mondes JS et Python	Non, écarté pour éviter la complexité d'un pont JS ↔ Python permanent

Conclusion : un **backend Python** est privilégié pour rester cohérent avec : - la plateforme data et les pipelines ETL majoritairement en Python ; - le socle IA également en Python

(FastAPI) ; ce qui simplifie les échanges entre services et réduit les risques d'incompréhension technique au sein de l'équipe.

7.3 3. Frontend

7.3.1 3.1 Next.js vs SPA React classique

Option	Avantages principaux	Limites / remarques	Choix retenu
Next.js	SSR/SSG, très bonne intégration avec React, BFF intégré (Next Server), gestion des Server Actions, performance et SEO	Architecture un peu plus avancée à appréhender	Oui – idéal pour un front web moderne consommant des APIs REST
React SPA classique	Simplicité de mise en place, très répandu, courbe d'apprentissage déjà connue par beaucoup d'étudiants	Pas de SSR natif, SEO plus faible, logique sensible côté client	Non, moins adapté aux besoins de performance et de sécurité

Conclusion : Next.js est retenu pour bénéficier du **BFF** et du rendu côté serveur, ce qui améliore à la fois la performance perçue, la sécurité (logique côté serveur) et l'expérience utilisateur, en cohérence avec l'objectif de proposer une **interface web performante et sécurisée** détaillé dans 01-technological-choices.md.

7.4 4. IA & NLP

7.4.1 4.1 Framework de modélisation NLP

Option	Avantages principaux	Limites / remarques	Choix retenu
HuggingFace Transformers	Écosystème complet pour le NLP, modèles francophones disponibles (CamemBERT, Mistral, etc.), usage local sans coût ni transfert de données, forte intégration Python	Ressources GPU recommandées pour l'entraînement (gérable en inférence CPU pour le MVP)	Oui – aligné avec les contraintes de budget nul, de souveraineté des données et de cohérence Python
OpenAI API / Claude API	Zéro infrastructure IA, très rapide à intégrer	Coût à l'usage (incompatible avec un budget nul), données envoyées hors infrastructure (souveraineté)	Non, écarté pour des raisons de coût et de souveraineté des données politiques

Option	Avantages principaux	Limites / remarques	Choix retenu
spaCy	Léger, rapide en production, bon pour la NER et le parsing	Moins puissant pour la classification de textes et la génération de résumés	Non, insuffisant pour les cas d'usage de résumés et de classification retenus

Conclusion : HuggingFace Transformers est retenu comme **socle de modélisation NLP** pour POPolitics, en cohérence avec les cas d'usage définis dans 01-technological-choices.md (section 8.2 : résumés automatiques, classification des textes politiques, analyse de tendances). Il garantit un usage local des modèles francophones, sans coût et sans transfert de données, aligné avec la contrainte de **souveraineté numérique** déjà mise en avant pour le choix de Kestra.

7.4.2 4.2 Exposition du service IA

Option	Avantages principaux	Limites / remarques	Choix retenu
FastAPI (IA Service dédié)	Léger, asynchrone, typage moderne, découplage propre entre IA Service et backend Django	Service supplémentaire à maintenir, géré via Docker Compose	Oui – conforme à l'architecture multi-services décrite dans 01-technological-choices.md
IA embarquée dans Django	Moins de services à orchestrer	Couplage fort entre logique IA et logique métier, contredit l'architecture multi-services	Non, écarté pour préserver la séparation des responsabilités

Conclusion : FastAPI est retenu comme **IA Service dédié**, exposé via une API REST au backend Django. Ce choix acte formellement le découplage décrit dans 01-technological-choices.md (sections 8.3 et 9.1), et lève l'ambiguïté laissée dans ce document quant à la technologie d'exposition des modèles IA.

7.5 5. Synthèse

- **ETL & Data : Kestra** pour l'orchestration des pipelines vers le Data Lake (Bronze / Silver / Gold) et les datamarts PostgreSQL, garantissant traçabilité et reproductibilité des traitements.
- **Backend : Django / Python** comme framework complet pour les services Data, IA et Auth, fortement aligné avec les pipelines ETL et les modèles IA développés en Python.

- **Frontend : Next.js** pour un frontend moderne, performant et sécurisé (BFF + SSR/SSG) offrant une expérience claire et accessible aux utilisateurs finaux.
- **IA & NLP : HuggingFace Transformers** pour la modélisation NLP locale (CamemBERT, Mistral), exposé via un **FastAPI IA Service** dédié consommé par le backend Django.

Ces choix prolongent les principes exposés dans 01-technological-choices.md : **séparation claire des responsabilités**, scalabilité progressive (MVP local → cloud), et alignement avec les bonnes pratiques de data engineering et de développement web modernes.

8 POPolitics – Ressources matérielles & coûts

Ce document complète les choix techniques de POPolitics en détaillant les ressources matérielles nécessaires, les coûts estimés et la logique d'évolution de l'infrastructure : - MVP en local via Docker sur machine de développement - V1 sur serveur local (on-premise léger) - Version Cloud managée

8.1 1. MVP local – Docker sur machine de développement

- **Contexte** : exécution en local sur la machine d'un membre de l'équipe (ou d'une machine partagée ponctuellement) avec l'ensemble des services Docker (Django, Next.js, PostgreSQL, Kestra, etc.).
 - **Configuration cible** :
 - CPU : 4 vCPU
 - RAM : 16 Go
 - Stockage : 256–512 Go SSD
 - **Coût estimatif** :
 - Utilisation d'un laptop/PC déjà existant → **0 € de coût supplémentaire** pour le projet.
 - **Justification** :
 - Suffisant pour développer, lancer les containers Docker et exécuter les premiers pipelines ETL et modèles IA sur des volumes réduits.
 - Permet de démarrer le projet sans frais, en cohérence avec la contrainte « budget nul au départ ».
-

8.2 2. V1 – Serveur local (on-premise léger)

- **Contexte** : hébergement sur un petit serveur local ou une machine dédiée dans l'école / l'organisation, toujours via Docker Compose.
- **Configuration cible** :

- CPU : 4–8 vCPU
 - RAM : 16–32 Go
 - Stockage : 512 Go–1 To SSD (pour Data Lake + datamarts + logs)
 - **Coût estimatif :**
 - Réutilisation d'un serveur existant → **0–200 €** (maintenance mineure, disques supplémentaires éventuels).
 - Achat d'un serveur d'entrée de gamme → **800–1 200 €** (one-shot, amortissable sur plusieurs projets).
 - **Justification :**
 - Offre un environnement plus stable pour les démos internes et tests d'intégration.
 - Permet d'augmenter la volumétrie manipulée sans saturer les machines personnelles.
-

8.3 3. Version Cloud – Hébergement managé

- **Contexte :** déploiement sur un fournisseur Cloud (type AWS, GCP, Azure ou équivalent souverain) avec une machine virtuelle ou un petit cluster de services managés.
 - **Configuration cible (exemple VM unique) :**
 - CPU : 4 vCPU
 - RAM : 16 Go
 - Stockage : 200–500 Go SSD (Data Lake et bases de données sur volumes attachés)
 - **Coût estimatif** (ordre de grandeur) :
 - VM 4 vCPU / 16 Go RAM : **40–80 €/mois** selon le fournisseur et la région.
 - Stockage : **5–20 €/mois** pour 200–500 Go.
 - **Justification :**
 - Permet une haute disponibilité, un accès distant pour tous les membres de l'équipe et une montée en charge progressive.
 - Facilite l'exposition sécurisée de l'application à des bêta-testeurs externes et son passage à l'échelle.
-

8.4 4. Cohérence avec les choix techniques

L'architecture et la stack retenues dans 01-technological-choices.md (Docker, Django, Next.js, PostgreSQL, Kestra, Data Lake Bronze/Silver/Gold) restent identiques dans ces trois scénarios : - on ne change que le niveau d'infrastructure (machine locale, serveur local, Cloud), - ce qui limite les coûts de migration entre MVP local, V1 sur serveur local et future version Cloud.

9 POPolitics – User Stories

9.1 1. Introduction

Ce document liste les **user stories** du projet POPolitics, organisées par **Epic** (grand domaine fonctionnel).

Chaque user story suit le format Scrum standard :

En tant que [persona], **je veux** [action] **afin de** [bénéfice].

Chaque story est accompagnée de :

- sa **priorité MoSCoW** (Must / Should / Could),
- ses **critères d'acceptation** (conditions à remplir pour considérer la story comme Done).

Note : les story points ne sont pas pré-assignés ici. Conformément au processus Scrum, ils sont **estimés collectivement par l'équipe lors des séances d'affinage du backlog** (voir [Méthodologie](#)).

9.2 2. Personas

Les personas détaillés (profil fictif complet, objectifs, frustrations, scénarios d'usage) sont documentés dans [00-personas.md](#).

Persona	Description	Profil fictif
Citoyen	Utilisateur grand public souhaitant suivre l'activité politique de ses représentants.	Marie Fontaine
Journaliste / Expert	Utilisateur averti cherchant à analyser des données politiques en profondeur.	Thomas Marchand
Administrateur	Membre de l'équipe POPolitics gérant la plateforme et les données.	Lucas Perrin

9.3 3. Épics

- [Epic 1 – Authentification & Gestion de compte](#)
- [Epic 2 – Portail collectif : Votes](#)
- [Epic 3 – Portail collectif : Tendances & Analyses](#)

- Epic 4 – Portail collectif : Synthèses IA
 - Epic 5 – Page personnelle : Suivi & Notifications
 - Epic 6 – Plateforme Data & ETL
-

9.4 Epic 1 – Authentification & Gestion de compte

9.4.1 US-101 – Créer un compte

En tant que citoyen, **je veux** créer un compte sur POPolitics **afin de** bénéficier d'une expérience personnalisée et accéder aux fonctionnalités protégées.

- **Priorité** : Must
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Je peux renseigner un email et un mot de passe pour créer mon compte.
 - Le mot de passe est soumis à une règle de complexité minimale (8 caractères, majuscule, chiffre).
 - Un message de confirmation est affiché après création réussie.
 - Il m'est impossible de créer deux comptes avec le même email.
-

9.4.2 US-102 – Se connecter

En tant que citoyen, **je veux** me connecter avec mes identifiants **afin d'** accéder à mon espace personnel et aux pages protégées.

- **Priorité** : Must
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Je peux me connecter avec email + mot de passe.
 - Une erreur claire s'affiche si les identifiants sont incorrects.
 - Je suis redirigé vers le Portail collectif après connexion réussie.
 - Les pages protégées sont inaccessibles sans être connecté.
-

9.4.3 US-103 – Se déconnecter

En tant que citoyen, **je veux** me déconnecter de la plateforme **afin de** sécuriser mon compte lorsque je quitte l'application.

- **Priorité** : Must
- **Story points** : *À estimer lors de l'affinage du backlog*
- **Critères d'acceptation** :

- Un bouton de déconnexion est accessible depuis toutes les pages authentifiées.
 - Après déconnexion, je suis redirigé vers la page de connexion.
 - Ma session est bien invalidée côté serveur.
-

9.4.4 US-104 – Réinitialiser son mot de passe

En tant que citoyen, **je veux** pouvoir réinitialiser mon mot de passe oublié **afin de** retrouver l'accès à mon compte.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Je peux saisir mon email pour recevoir un lien de réinitialisation.
 - Le lien de réinitialisation expire après 1 heure.
 - Je peux définir un nouveau mot de passe via ce lien.
-

9.5 Epic 2 – Portail collectif : Votes

9.5.1 US-201 – Consulter le tableau des votes

En tant que citoyen, **je veux** consulter un tableau des votes des élus **afin de** savoir comment mes représentants votent sur les textes de loi.

- **Priorité** : Must
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Le tableau affiche pour chaque vote : l' élu, le texte, l'institution (AN / Sénat / UE), la date et la position (Pour / Contre / Abstention / Non-votant).
 - Les données sont mises à jour au moins une fois par jour.
 - Un code couleur distinct est appliqué à chaque position.
-

9.5.2 US-202 – Filtrer les votes

En tant que citoyen, **je veux** filtrer les votes par institution, période, groupe politique et élu **afin de** retrouver rapidement les informations qui m'intéressent.

- **Priorité** : Must
- **Story points** : *À estimer lors de l'affinage du backlog*
- **Critères d'acceptation** :
 - Les filtres disponibles sont : institution, période (date début / fin), groupe politique, nom de l' élu.

- Les résultats se mettent à jour dynamiquement lors de l'application des filtres.
 - Il est possible de réinitialiser tous les filtres en un clic.
-

9.5.3 US-203 – Filtrer les votes par thème, commission et type de texte

En tant que journaliste, **je veux** filtrer les votes par thème, commission et type de texte **afin d'** affiner mes analyses sur un sujet précis.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Les filtres thème, commission et type de texte (amendement vs vote final) sont disponibles en complément des filtres de base.
 - Les filtres sont combinables entre eux.
-

9.5.4 US-204 – Consulter les indicateurs d'un vote

En tant que citoyen, **je veux** voir le taux de participation et la cohésion du groupe pour un vote donné **afin de** comprendre la dynamique politique derrière le résultat.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Le taux de participation (% de votants) est affiché pour chaque vote.
 - Un indice de cohésion de groupe (% de membres ayant voté dans le même sens) est calculé et affiché.
-

9.5.5 US-205 – Exporter les résultats de vote

En tant que journaliste, **je veux** exporter les données de vote au format CSV ou PDF **afin de** les réutiliser dans mes articles et analyses.

- **Priorité** : Could
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Un bouton d'export CSV et PDF est disponible sur le tableau des votes.
 - L'export respecte les filtres actifs au moment du téléchargement.
 - Le fichier exporté contient les colonnes : élu, texte, institution, date, position.
-

9.6 Epic 3 – Portail collectif : Tendances & Analyses

9.6.1 US-301 – Visualiser les indicateurs de cohérence d'un élu

En tant que citoyen, **je veux** visualiser l'indice de cohérence d'un élu avec son groupe politique **afin de** savoir s'il suit les positions de son parti ou s'en écarte.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Une jauge ou score de cohérence est affiché sur la fiche d'un élu.
 - Les votes atypiques (où l'élu a voté contre son groupe) sont mis en évidence.
-

9.6.2 US-302 – Comparer deux élus ou deux groupes

En tant que journaliste, **je veux** comparer les positions de deux élus ou de deux groupes politiques **afin d'** identifier leurs convergences et divergences.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Je peux sélectionner deux élus ou deux groupes pour lancer une comparaison.
 - Une vue côte-à-côte affiche leurs positions respectives sur les mêmes textes.
 - Un score de similarité globale est calculé et affiché.
-

9.6.3 US-303 – Visualiser la cartographie des alliances

En tant que journaliste, **je veux** voir une cartographie des alliances entre groupes politiques **afin d'** identifier quels groupes votent ensemble et sur quels sujets.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Un graphe de réseau affiche les groupes politiques et les liens entre eux pondérés par la fréquence d'alignement.
 - Il est possible de filtrer la cartographie par institution ou par thème.
-

9.6.4 US-304 – Croiser les données AN / Sénat / UE

En tant que expert, **je veux** croiser les données de vote entre l'Assemblée nationale, le Sénat et le Parlement européen **afin d'** analyser la cohérence des positions politiques entre institutions.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l’affinage du backlog*
 - **Critères d’acceptation** :
 - Le datamart Cross-match est disponible et interrogeable via l’interface.
 - Je peux filtrer les résultats par élu ou groupe pour voir ses positions sur plusieurs institutions.
-

9.7 Epic 4 – Portail collectif : Synthèses IA

9.7.1 US-401 – Obtenir un résumé automatique d’un débat

En tant que citoyen, **je veux** obtenir un résumé automatique d’un débat parlementaire **afin de** comprendre l’essentiel en moins d’une minute sans lire le transcript complet.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l’affinage du backlog*
 - **Critères d’acceptation** :
 - Un bouton “Résumé IA” est accessible sur la page d’un débat.
 - Le résumé est présenté en 5 points clés maximum.
 - Le temps de génération est inférieur à 5 secondes.
 - Un lien vers le transcript complet est proposé sous le résumé.
-

9.7.2 US-402 – Consulter une fiche éclair sur un texte de loi

En tant que citoyen, **je veux** accéder à une fiche éclair sur un projet de loi ou amendement **afin de** comprendre son objectif, ses enjeux et les positions des acteurs en quelques secondes.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l’affinage du backlog*
 - **Critères d’acceptation** :
 - La fiche présente 3 vues : Résumé / Enjeux / Acteurs.
 - La section Acteurs indique qui est pour et qui est contre, avec une courte justification.
 - La fiche est accessible depuis le tableau des votes.
-

9.8 Epic 5 – Page personnelle : Suivi & Notifications

9.8.1 US-501 – Suivre des élus

En tant que citoyen, **je veux** sélectionner des élus à suivre **afin d’**être informé de leurs actions et votes sans avoir à les chercher à chaque visite.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l’affinage du backlog*
 - **Critères d’acceptation** :
 - Je peux rechercher un élu par nom et l’ajouter à ma liste de suivi.
 - Ma liste de suivi est persistante entre les sessions.
 - Je peux retirer un élu de ma liste de suivi à tout moment.
-

9.8.2 US-502 – Consulter la timeline d’un élu suivi

En tant que citoyen, **je veux** consulter la timeline des actions d’un élu que je suis **afin de** voir d’un coup d’œil ses votes récents, interventions et amendements.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l’affinage du backlog*
 - **Critères d’acceptation** :
 - La timeline affiche les événements des élus suivis dans l’ordre chronologique inverse.
 - Chaque événement indique le type (vote, intervention, amendement), la date et un bref libellé.
 - Je peux cliquer sur un événement pour accéder au détail.
-

9.8.3 US-503 – Configurer ses thèmes d’intérêt

En tant que citoyen, **je veux** choisir des thèmes politiques qui m’intéressent (économie, environnement, social...) **afin de** recevoir uniquement les alertes pertinentes pour moi.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l’affinage du backlog*
 - **Critères d’acceptation** :
 - Une liste de thèmes prédéfinis est proposée lors du paramétrage du profil.
 - Je peux en sélectionner plusieurs.
 - Mes préférences de thèmes peuvent être modifiées à tout moment depuis mon profil.
-

9.8.4 US-504 – Recevoir des notifications personnalisées

En tant que citoyen, **je veux** recevoir des notifications sur les événements importants liés à mes élus suivis et à mes thèmes d'intérêt **afin d'** être informé sans être submergé d'alertes.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Je ne reçois des notifications que pour les événements liés à mes élus suivis ou à mes thèmes d'intérêt.
 - Chaque notification affiche un libellé clair et un lien direct vers la page concernée.
 - Je peux désactiver certaines catégories de notifications depuis mes paramètres.
-

9.9 Epic 6 – Plateforme Data & ETL

9.9.1 US-601 – Ingérer les données de l'Assemblée nationale

En tant qu' administrateur, **je veux** que les données de vote de l'AN soient ingérées automatiquement **afin de** disposer d'une base à jour sans intervention manuelle.

- **Priorité** : Must
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Le pipeline ETL AN s'exécute automatiquement selon une fréquence définie (quotidienne minimum).
 - Les données brutes sont stockées en zone Bronze.
 - Les exécutions sont tracées (logs, statut, horodatage) dans Kestra.
 - En cas d'échec, une alerte est émise et le job peut être relancé manuellement.
-

9.9.2 US-602 – Ingérer les données du Sénat

En tant qu' administrateur, **je veux** que les données de vote du Sénat soient ingérées automatiquement **afin de** couvrir les deux chambres du Parlement français.

- **Priorité** : Must
- **Story points** : *À estimer lors de l'affinage du backlog*
- **Critères d'acceptation** :
 - Identiques à US-601, appliquées à la source Sénat.

- Les données Sénat sont stockées dans un datamart PostgreSQL distinct et documenté.

9.9.3 US-603 – Ingérer les données du Parlement européen

En tant qu’ administrateur, je veux que les données du Parlement européen soient ingérées automatiquement **afin d’** offrir une dimension européenne à l’analyse.

- **Priorité** : Must
- **Story points** : *À estimer lors de l’affinage du backlog*
- **Critères d’acceptation** :
 - Identiques à US-601, appliquées à la source UE.
 - Les données UE sont stockées dans un datamart PostgreSQL distinct et documenté.

9.9.4 US-604 – Normaliser les données en zone Silver

En tant qu’ administrateur, je veux que les données brutes soient nettoyées et normalisées en zone Silver **afin de** disposer d’une base cohérente et exploitable par le backend.

- **Priorité** : Must
- **Story points** : *À estimer lors de l’affinage du backlog*
- **Critères d’acceptation** :
 - Les données Silver respectent un schéma normalisé documenté.
 - Les doublons et données incohérentes sont traités et tracés.
 - La transformation Bronze → Silver est reproductible (mêmes entrées → mêmes sorties).

9.9.5 US-605 – Préparer les données analytiques en zone Gold

En tant qu’ administrateur, je veux que des données agrégées et prêtes à l’analyse soient disponibles en zone Gold **afin de** réduire les temps de requête du backend et enrichir les indicateurs.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l’affinage du backlog*
 - **Critères d’acceptation** :
 - La zone Gold contient des tables pré-agrégées (indicateurs, KPI) consommables directement par les APIs.
 - La transformation Silver → Gold est orchestrée par Kestra.
-

9.10 4. Récapitulatif

Les story points sont laissés vides et seront renseignés lors des séances d'affinage du backlog.

ID	Titre court	Epic	Priorité	Points
US-101	Créer un compte	Auth	Must	
US-102	Se connecter	Auth	Must	
US-103	Se déconnecter	Auth	Must	
US-104	Réinitialiser mot de passe	Auth	Should	
US-201	Consulter tableau des votes	Votes	Must	
US-202	Filtrer les votes (base)	Votes	Must	
US-203	Filtrer les votes (avancé)	Votes	Should	
US-204	Indicateurs d'un vote	Votes	Should	
US-205	Exporter les votes	Votes	Could	
US-301	Cohérence d'un élu	Tendances	Should	
US-302	Comparer élus / groupes	Tendances	Should	
US-303	Cartographie des alliances	Tendances	Should	
US-304	Croisement AN / Sénat / UE	Tendances	Should	
US-401	Résumé IA d'un débat	IA	Should	
US-402	Fiche éclair sur un texte	IA	Should	
US-501	Suivre des élus	Page perso	Should	
US-502	Timeline d'un élu suivi	Page perso	Should	
US-503	Configurer thèmes d'intérêt	Page perso	Should	
US-504	Notifications personnalisées	Page perso	Should	
US-601	Ingestion données AN	Data	Must	
US-602	Ingestion données Sénat	Data	Must	
US-603	Ingestion données UE	Data	Must	
US-604	Normalisation zone Silver	Data	Must	
US-605	Agrégation zone Gold	Data	Should	

10 POPolitics - Plan de Sprints

10.1 1. Introduction

Ce document décrit le **decoupage du projet en sprints**, aligne sur le rythme d'alternance Master (voir [Methodologie](#)).

10.1.1 Principes

- Les sprints ont une **duree fixe de 9 semaines**.
- Le planning démarre le **21/09/2026** et se termine le **25/06/2027**.
- Les semaines de presentiel sont integrees dans les sprints en cours.
- Le dernier sprint peut etre plus court que 9 semaines pour s'aligner sur la date de fin d'annee.
- Le contenu exact de chaque sprint (user stories, story points) est valide lors du Sprint Planning et affine lors des seances d'affinage du backlog.

10.2 2. Vue d'ensemble

Periode	Dates	Duree	Objectif global
Sprint 1	21/09/2026 -> 20/11/2026	9 semaines	Fondations (Auth + ETL)
Sprint 2	23/11/2026 -> 22/01/2027	9 semaines	MVP Portail collectif
Sprint 3	25/01/2027 -> 26/03/2027	9 semaines	Enrichissements (tendances + page perso)
Sprint 4	29/03/2027 -> 28/05/2027	9 semaines	IA + finitions
Sprint 5	31/05/2027 -> 25/06/2027	4 semaines	Stabilisation finale + soutenance

10.3 3. Detail des sprints

10.3.1 Sprint 1 - Fondations (Must)

Duree	9 semaines
Dates	21/09/2026 -> 20/11/2026
Sprint Goal	Poser les fondations techniques: authentication operationnelle et pipelines ETL de base.

User stories cibles :

ID	Titre	Priorite
US-101	Creer un compte	Must
US-102	Se connecter	Must
US-103	Se deconnecter	Must

ID	Titre	Priorite
US-601	Ingestion donnees AN	Must
US-602	Ingestion donnees Senat	Must
US-603	Ingestion donnees UE	Must
US-604	Normalisation zone Silver	Must

10.3.2 Sprint 2 - MVP Portail collectif

Duree	9 semaines
Dates	23/11/2026 -> 22/01/2027
Sprint Goal	Livrer un portail collectif utilisable avec donnees reelles, filtres et premiers indicateurs.

User stories cibles :

ID	Titre	Priorite
US-201	Consulter tableau des votes	Must
US-202	Filtrer les votes (base)	Must
US-605	Agregation zone Gold	Should
US-203	Filtrer les votes (avance)	Should
US-204	Indicateurs d'un vote	Should

10.3.3 Sprint 3 - Enrichissements (Should)

Duree	9 semaines
Dates	25/01/2027 -> 26/03/2027
Sprint Goal	Ajouter les fonctionnalites d'analyse politique et les usages personnalisés.

User stories cibles :

ID	Titre	Priorite
US-301	Coherence d'un élu	Should
US-302	Comparer élus / groupes	Should
US-304	Croisement AN / Senat / UE	Should
US-501	Suivre des élus	Should
US-502	Timeline d'un élu suivi	Should
US-503	Configurer themes d'interet	Should

ID	Titre	Priorite
US-504	Notifications personnalisées	Should

10.3.4 Sprint 4 - IA et finitions (Should/Could)

Duree	9 semaines
Dates	29/03/2027 -> 28/05/2027
Sprint Goal	Integrer la couche IA, finaliser les visualisations avancees et traiter les finitions.

User stories cibles :

ID	Titre	Priorite
US-401	Resume IA d'un debat	Should
US-402	Fiche éclair sur un texte	Should
US-303	Cartographie des alliances	Should
US-104	Reinitialiser mot de passe	Should
US-205	Exporter les votes	Could

10.4 4. Sprint 5 - Stabilisation & soutenance

Duree	4 semaines
Dates	31/05/2027 -> 25/06/2027
Objectif	Stabiliser, corriger, preparer la soutenance et finaliser les livrables academiques.

Travaux prevus :

- Corrections de bugs transverses.
- Tests de bout en bout sur les parcours critiques.
- Preparation du scenario de demonstration.
- Mise a jour finale de la documentation technique.
- Preparation des slides de soutenance.

10.5 5. Recapitulatif

Periode	Duree	Stories Must	Stories Should	Stories Could
Sprint 1	9 sem.	7	0	0
Sprint 2	9 sem.	2	3	0

Periode	Duree	Stories Must	Stories Should	Stories Could
Sprint 3	9 sem.	0	7	0
Sprint 4	9 sem.	0	4	1
Sprint 5	4 sem.	-	-	-

Total stories Must Have couvertes : 9 / 9

Total stories Should Have couvertes : 14 / 14

Total stories Could Have couvertes : 1 / 1

11 POPolitics – Registre des risques

11.1 1. Méthode d'évaluation

Chaque risque est évalué selon deux axes :

- **Probabilité** : Faible (1) / Moyen (2) / Élevé (3)
- **Impact** : Faible (1) / Moyen (2) / Élevé (3)
- **Criticité** = Probabilité × Impact → score de 1 à 9

Criticité	Niveau
1 – 2	Faible
3 – 4	Moyen
6 – 9	Critique

11.2 2. Registre des risques

11.2.1 Catégorie : Données & Sources externes

ID	Risque	Probabilité	Impact	Criticité	Mitigation	Responsable
R-01	Change ment de format ou d'URL d'une API source (AN, Sénat, UE) sans préavis	2	3	6	Version ner les connecteurs ETL ; surveiller les changelogs des portails open data ; couche	Data Engineer

ID	Risque	Probabilité	Impact	Criticité	Mitigation	Responsable
					Bronze comme filet de sécurité	
R-02	Qualité des données insuffisante (doublets, valeurs manquantes, incohérences entre sources)	3	2	6	Tests de qualité systématiques en phase Silver ; logs de rejet explicites dans Kestra ; validation d'un échantillon à chaque ingestion	Data Engineer
R-03	Indisponibilité temporaire d'une source (maintenance portail, panne)	2	2	4	Gestion des runs vides dans Kestra (no-op) ; retry automatique ; stocker les dernières données	Data Engineer

ID	Risque	Probabilité	Impact	Criticité	Mitigation	Responsable
					valides en Bronze	
R-04	Absence d'identification commune entre AN, Sénat et UE pour le cross-match	3	2	6	Table de correspondance manuelle (nom + prénom + date de naissance) ; documenter les cas ambigus ; limiter le cross-match aux élus avec mandat simultané	Data Engineer
R-05	Volume de données plus important que prévu (saturation disque)	1	2	2	Surveiller la taille des données dès le Sprint 1 ; prévoir un	DevOps

ID	Risque	Probabilité	Impact	Criticité	Mitigation	Responsable
	ou mémoire)				quota disque minimum (50 Go) sur la VM de démo	

11.2.2 Catégorie : Technique & Architecture

ID	Risque	Probabilité	Impact	Criticité	Mitigation	Responsable
R-06	Complexité de la stack locale (7-8 conteneurs Docker) bloquant le démarrage d'un membre	2	2	4	Guide de démarrage rapide obligatoire dès le Sprint 0 ; critère de fin de Sprint 0 : stack opérationnelle en < 30 min pour chacun	Tech Lead
R-07	Couche IA (Sprint 9) sous-estimée —	3	3	9	Réduire le périmètre IA au strict minimum	Tech Lead / IA

ID	Risque	Probabilité	Impact	Criticité	Mitigation	Responsable
	modèle non livrable dans les temps				m (résumé automatique unique ment) ; prévoir un fallback sans IA si nécessaire ; anticiper en Sprint 8	
R-08	Divergence des schémas de données entre AN et Sénat rendant la normalisation Silver complexe	3	2	6	Documenter les deux schémas dès le Sprint 1 ; définir le schéma unifié avant le Sprint 2 ; tester la couche Silver avec données réelles	Data Engineer
R-09	Régression	2	3	6	Tests unitaire	Backend / Data

ID	Risque	Probabilité	Impact	Criticité	Mitigation	Responsable
	backen d ou ETL non détecté e faute de tests automa tisés suffisan ts				s sur les transfor mations critique s ; tests d'intégr ation backen d ↔ datama rts ; smoke tests avant chaque démon	
R-10	Perform ance insuffis ante des requête s Postgre SQL sur de gros volume s	2	2	4	Indexer les colonne s clés dès la créatio n des datama rts ; surveill er les temps de réponse des endpoin ts ; optimis er en Gold si nécess aire	Backen d / Data

11.2.3 Catégorie : Organisation & Équipe

ID	Risque	Probabilité	Impact	Criticité	Mitigation	Responsable
R-11	Manque de disponibilité de membres en période de distanciel (alternance, charge entreprise)	3	2	6	Sprints de 2 semaines avec objectifs réalistes ; revue de charge lors du Sprint Planning ; périmètre ajustable (Should → Could)	Scrum Master / PO
R-12	Départ ou indisponibilité prolongée d'un membre clé (maladie, abandon)	1	3	3	Documentation continue des choix et décisions ; pas de compétence en silo ; pair programming encouragé	Tech Lead

ID	Risque	Probabilité	Impact	Criticité	Mitigation	Responsable
R-13	Mauvaise synchronisation entre équipes data, backend et frontend (dépendances bloquantes)	2	3	6	Définir les contrats d'API et les schémas de données avant le début du Sprint concerné ; daily ou point de synchro hebdomadaire	Scrum Master
R-14	Sous-estimation générale de la charge sur un sprint	2	2	4	Vélocité suivie sprint par sprint ; affinage du backlog régulier ; règle : mieux vaut finir moins et bien que livrer un sprint instable	PO / Scrum Master

11.2.4 Catégorie : Périmètre & Livraison

ID	Risque	Probabilité	Impact	Criticité	Mitigation	Responsable
R-15	Périmètre Must Have non livré avant la soutenance	1	3	3	Prioriser strictement le Must Have ; bloquer toute fonctionnalité Should si un Must est en retard	PO
R-16	Scénario de démonstration non préparé ou instable le jour J	2	3	6	Dédier le Sprint 11 à la stabilisation et à la préparation de la démo ; jeu de données de démonstration figé et testé à l'avance	Tech Lead / PO
R-17	Documentation technique	2	2	4	Mise à jour des docs incluse dans la	Toute l'équipe

ID	Risque	Probabilité	Impact	Criticité	Mitigation	Responsable
	incomplète ou désynchronisé avec le code				Definition of Done ; revue documentation lors du Sprint 11	

11.3 3. Récapitulatif par criticité

11.3.1 Critiques (score 6–9)

ID	Risque	Score
R-07	Couche IA sous-estimée	9
R-01	Changement d'API source	6
R-02	Qualité des données insuffisante	6
R-04	Pas d'identifiant commun cross-match	6
R-08	Divergence schémas AN / Sénat	6
R-09	Régressions non détectées	6
R-11	Disponibilité en distanciel	6
R-13	Désynchronisation équipes	6
R-16	Démo instable le jour J	6

11.3.2 Moyens (score 3–4)

ID	Risque	Score
R-06	Stack locale bloquante	4
R-10	Performance PostgreSQL	4
R-14	Sous-estimation de charge	4
R-17	Documentation désynchronisée	4
R-12	Départ d'un membre clé	3
R-15	Must Have non livré	3

11.3.3 Faibles (score 1–2)

ID	Risque	Score
----	--------	-------

ID	Risque	Score
R-05	Volume de données excessif	2

11.4 4. Suivi

Ce registre est à réviser à **chaque Sprint Review**. Pour chaque risque critique actif, une ligne de suivi doit être ajoutée lors de la révision :

ID	Date révision	Statut	Évolution
----	---------------	--------	-----------

Statuts possibles : Actif / Atténué / Clos / Matérialisé

12 POPolitics – Exigences non-fonctionnelles (NFR)

12.1 1. Introduction

Les exigences non-fonctionnelles définissent **comment** le système doit se comporter, indépendamment des fonctionnalités métier. Elles s'appliquent à l'ensemble de la stack : pipeline ETL, backend Django, frontend Next.js et infrastructure Docker.

12.2 2. Performance

12.2.1 2.1 Temps de réponse API

Endpoint	Cible MVP	Cible V1
Consultation tableau des votes (filtre simple)	< 2 s	< 1 s
Consultation tableau des votes (filtre avancé / cross-match)	< 5 s	< 2 s
Page personnelle (timeline d'un élu suivi)	< 3 s	< 1,5 s
Authentification (login)	< 1 s	< 500 ms
Endpoint IA (résumé automatique)	< 10 s	< 5 s

12.2.2 2.2 Pipeline ETL

- Ingestion complète AN + Sénat + UE : < **2 heures** pour un run journalier complet.
- Ingestion différentielle (données du jour uniquement) : < **30 minutes**.

- Kestra doit loguer le temps d'exécution de chaque step pour permettre l'identification des goulots d'étranglement.

12.2.3 2.3 Chargement frontend

- **LCP (Largest Contentful Paint)** : < 2,5 s sur connexion standard.
- **TTI (Time to Interactive)** : < 4 s.
- Les pages lourdes (tableaux, graphiques) doivent implémenter une **pagination** ou un **chargement progressif** pour ne jamais dépasser 500 lignes chargées en une seule requête.

12.3 3. Scalabilité

12.3.1 3.1 Charge utilisateurs

Phase	Utilisateurs simultanés cibles
MVP local (démonstration)	5–10
V1 serveur local	50–100
Version Cloud	500+

12.3.2 3.2 Volume de données

Source	Volume estimé (initial)	Croissance annuelle
Assemblée nationale	~5 Go (historique depuis 2007)	~500 Mo/an
Sénat	~3 Go (historique depuis 2010)	~300 Mo/an
Parlement européen	~2 Go (historique depuis 2004)	~200 Mo/an
Total	~10 Go	~1 Go/an

- L'architecture Bronze/Silver/Gold doit rester fonctionnelle jusqu'à **100 Go** sans modification structurelle.
- Le passage à un stockage Cloud (S3 ou équivalent) doit être possible sans refonte du pipeline ETL.

12.4 4. Disponibilité & fiabilité

12.4.1 4.1 Disponibilité cible

Environnement	Disponibilité cible
MVP local (démonstration)	Best-effort (pas de SLA)
V1 serveur local	95 % (tolérance maintenance planifiée)
Version Cloud	99 %

12.4.2 4.2 Gestion des pannes

- Un job ETL en échec ne doit **pas bloquer les autres jobs**, isolation par pipeline dans Kestra.
- En cas d'indisponibilité d'une source externe, le système doit servir les **dernières données valides** sans erreur visible pour l'utilisateur.
- Le backend Django doit retourner des **codes HTTP explicites** (404, 503, etc.) et jamais exposer de stack trace en production.

12.4.3 4.3 Reprise après incident

- Temps de redémarrage de la stack complète (Docker Compose) : **< 5 minutes**.
 - Les pipelines Kestra doivent supporter le **rejeu manuel** d'un run en échec sans duplication de données.
-

12.5 5. Sécurité

12.5.1 5.1 Authentification & autorisation

- Authentification via **JWT** ou session sécurisée (HttpOnly cookie).
- Toute route API Django doit vérifier l'authentification **avant** tout traitement.
- Les rôles (utilisateur standard, administrateur) doivent être vérifiés côté serveur, jamais côté client seul.
- Les tokens JWT doivent avoir une **durée de vie courte** (< 1 heure) avec mécanisme de refresh.

12.5.2 5.2 Protection des données

- Aucun secret (clés API, mots de passe, tokens) ne doit être commité dans le dépôt Git.
- Les secrets sont gérés via **variables d'environnement** et/ou fichier .env non versionné.
- Les mots de passe utilisateurs sont stockés **hashés** (bcrypt ou argon2), jamais en clair.

12.5.3 5.3 Sécurité des échanges

- Toutes les communications entre frontend et backend doivent transiter en **HTTPS** (dès la V1 serveur).
- Les APIs doivent implémenter une protection **CORS** explicite (whitelist des origines autorisées).
- Protection contre les injections SQL via l'ORM Django — pas de requêtes SQL concaténées manuellement.

12.5.4 5.4 Données personnelles (RGPD)

- Les données traitées sont des **données publiques d'élus** dans l'exercice de leur mandat, pas de contrainte RGPD spécifique sur ces données.
 - Les données des **utilisateurs inscrits** (email, préférences) sont soumises au RGPD :
 - Consentement explicite à l'inscription.
 - Possibilité de suppression du compte et des données associées.
 - Pas de partage avec des tiers.
-

12.6 6. Maintenabilité

12.6.1 6.1 Lisibilité du code

- Couverture de tests unitaires sur les fonctions critiques : > **60 %** (cible MVP).
- Chaque pipeline Kestra doit être nommé explicitement et documenté en en-tête YAML.
- Pas de valeurs en dur (*hardcoded*) dans les scripts ETL, utiliser des variables de configuration.

12.6.2 6.2 Traçabilité

- Chaque run ETL doit produire un log structuré contenant : date, source, nombre de lignes ingérées, nombre de rejets, statut final.
- Les erreurs backend doivent être loguées avec leur contexte (endpoint, paramètres, timestamp), sans données sensibles.

12.6.3 6.3 Évolutivité de l'architecture

- L'ajout d'une nouvelle source de données (ex. conseils régionaux) doit être possible **sans modifier** les couches Silver/Gold existantes.
 - L'ajout d'un nouveau modèle IA doit se faire via un nouveau service indépendant, sans modifier le backend Django existant.
-

12.7 7. Compatibilité

12.7.1 7.1 Navigateurs supportés

Navigateur	Support
Chrome / Chromium (dernière version)	Complet
Firefox (dernière version)	Complet
Safari (dernière version)	Complet
Edge (dernière version)	Complet
Internet Explorer	Non supporté

12.7.2 7.2 Résolutions

- L'interface doit être utilisable sur **desktop** (largeur ≥ 1024 px) en priorité.
- Une adaptation **tablette** (≥ 768 px) est un objectif Should Have.
- Le support mobile n'est pas dans le périmètre du MVP.

12.8 8. Contraintes d'infrastructure (MVP)

En cohérence avec [08-projet-ressources.md](#) :

- Stack complète opérationnelle sur une machine **4 vCPU / 16 Go RAM / 256 Go SSD**.
- Démarrage de la stack via docker-compose up en **< 5 minutes** sur la machine de démo.
- Aucun service cloud payant requis pour le MVP — tout doit fonctionner **offline** (sauf les appels aux sources externes lors des runs ETL).

13 POPolitics – Sources de données

13.1 1. Vue d'ensemble

POPolitics s'appuie exclusivement sur des **données publiques ouvertes** issues de trois institutions :

Institution	Portail open data	Licence
Assemblée nationale	data.assemblee-nationale.fr	Licence Ouverte Etalab 2.0
Sénat	data.senat.fr	Licence Ouverte Etalab 2.0
Parlement européen	data.europarl.europa.eu	Licence CC BY 4.0

Toutes les sources sont **gratuites**, sans clé d'API requise pour l'accès de base.

13.2 2. Assemblée nationale

13.2.1 2.1 Portail

- URL : <https://data.assemblee-nationale.fr>
- API REST (Recherche AN) : <https://recherche.assemblee-nationale.fr/api>

13.2.2 2.2 Données disponibles

Dataset	Description	Format
Scrutins / votes	Résultats des votes en séance publique	JSON, XML

Dataset	Description	Format
Députés	Identité, groupe politique, circonscription, mandats	JSON, XML
Amendements	Dépôt, sort, auteur, texte de référence	JSON, XML
Dossiers législatifs	Textes de loi, étapes, commissions	JSON, XML
Agenda	Ordre du jour des séances plénières	JSON, iCal
Comptes rendus	Verbatim des débats en séance	HTML, PDF

13.2.3 2.3 Fréquence de mise à jour

- Votes et dossiers : **quotidienne** (jours de séance).
- Référentiels élus : **à chaque changement** (élection partielle, démission, décès).
- Comptes rendus : **sous 24–48h** après la séance.

13.2.4 2.4 Particularités

- Les identifiants d'élus (acteur_uid) sont stables entre les législatures.
- Les scrutins contiennent le vote individuel de chaque député (pour / contre / abstention / absent).
- Les données remontent jusqu'à la **13e législature** (2007) selon les datasets.

13.3 3. Sénat

13.3.1 3.1 Portail

- URL : <https://data.senat.fr>
- API REST : <https://api.senat.fr>

13.3.2 3.2 Données disponibles

Dataset	Description	Format
Scrutins / votes	Votes en séance publique et en commission	JSON, XML
Sénateurs	Identité, groupe, département, mandats	JSON, XML
Amendements	Dépôt, sort, co-signataires	JSON, XML
Dossiers législatifs	Textes, navettes, commissions	JSON, XML
Agenda	Calendrier des séances	JSON

Dataset	Description	Format
Comptes rendus	Verbatim des débats	HTML, PDF

13.3.3 3.3 Fréquence de mise à jour

- Votes : **sous 24h** après la séance.
- Référentiels sénateurs : **temps quasi réel**.
- Comptes rendus : **48–72h** après la séance.

13.3.4 3.4 Particularités

- Le modèle de données Sénat est **structurellement différent** de celui de l'AN (schémas non identiques — transformation requise en phase Silver).
- Les votes en commission sont moins systématiquement disponibles que ceux en plénière.
- Certains anciens datasets (avant 2010) sont en **XML uniquement**.

13.4 4. Parlement européen

13.4.1 4.1 Portail

- URL : <https://data.europarl.europa.eu>
- API REST (SPARQL endpoint & REST) : <https://data.europarl.europa.eu/api/v1>

13.4.2 4.2 Données disponibles

Dataset	Description	Format
Votes en plénière	Résultats par eurodéputé	JSON-LD, RDF, XML
Eurodéputés (MEPs)	Identité, groupe politique, pays, mandats	JSON, XML
Activités parlementaires	Questions, rapports, résolutions	JSON, XML
Documents officiels	Textes législatifs (règlements, directives)	JSON, PDF
Agenda	Sessions plénières et ordre du jour	JSON, iCal

13.4.3 4.3 Fréquence de mise à jour

- Votes : **après chaque session plénière** (hebdomadaire à Strasbourg, mensuelle à Bruxelles).
- Référentiels élus : **à chaque changement** (rotation liée aux élections nationales ou décès).

13.4.4 4.4 Particularités

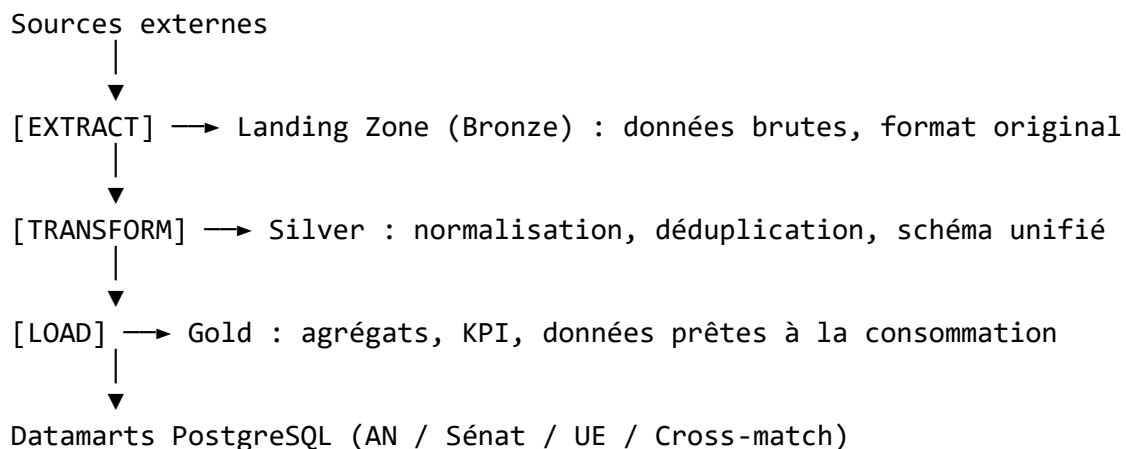
- L'API utilise partiellement le format **JSON-LD / RDF** (données liées) — parsing plus complexe.
- Les identifiants MEP (mepId) sont stables sur toute la mandature.
- Les données sont disponibles en **24 langues** ; nous ciblons le **français et l'anglais** uniquement.
- La couverture remonte à la **6e législature** (2004).

13.5 5. Formats et ingestion

13.5.1 5.1 Formats bruts attendus

Format	Sources concernées	Traitement requis
JSON	AN, Sénat, UE	Parsing direct
XML	AN, Sénat (anciens datasets)	Conversion JSON ou parsing XML
JSON-LD / RDF	UE (API SPARQL)	Parsing spécifique, aplatissement
HTML	Comptes rendus AN, Sénat	Scraping + nettoyage texte
PDF	Comptes rendus, textes officiels	Extraction texte (pdfplumber ou équivalent)

13.5.2 5.2 Stratégie d'ingestion (pipeline ETL)



13.5.3 5.3 Identifiants croisés

Pour le datamart **Cross-match**, le rapprochement entre institutions s'appuie sur :

- **Nom + prénom + date de naissance** (aucun identifiant commun inter-institutionnel officiel).

- **Groupe politique européen** ↔ **groupe parlementaire français** (table de correspondance à maintenir manuellement).

13.6 6. Licences et conditions d'utilisation

Source	Licence	Conditions
Assemblée nationale	Licence Ouverte Etalab 2.0	Mention de la source obligatoire
Sénat	Licence Ouverte Etalab 2.0	Mention de la source obligatoire
Parlement européen	CC BY 4.0	Mention de la source + intégrité des données

- **Utilisation commerciale** : autorisée sous ces licences.
- **Modification / redistribution** : autorisées, sous réserve de mention.
- **Données personnelles** : les données des élus sont des données publiques dans l'exercice de leur mandat — pas de contrainte RGPD particulière pour leur traitement dans ce contexte.

13.7 7. Limitations connues

Limitation	Source(s)	Impact	Mitigation
Historique partiel (< 2007 pour l'AN)	AN	Analyses longue durée limitées	Cadrer les analyses à partir de 2007
Schémas non normalisés entre AN et Sénat	AN, Sénat	Complexité de la couche Silver	Table de mapping explicite
Format JSON-LD complexe	UE	Temps de parsing plus long	Librairie RDFLib ou conversion préalable
Comptes rendus en PDF/HTML	AN, Sénat	Extraction texte fragile	Prétraitement dédié, validation manuelle d'un échantillon
Absences non distinguées	AN	Vote "absent" ≠ "ne participe pas"	Annoter dans le schéma Silver
Mises à jour irrégulières hors sessions	Toutes	Pipelines vides certains jours	Gestion des runs vides dans Kestra (no-op)

13.8 8. Évolutions envisagées

- **Could have** : ajout d'autres sources (conseils régionaux, assemblées d'outre-mer).
- **Won't have (for now)** : flux temps réel ou streaming — ingestion batch journalière suffisante pour le MVP.

14 POPolitics – Sécurité & RGPD

14.1 1. Introduction

Ce document décrit les mesures de sécurité applicatives et les obligations RGPD du projet POPolitics. Il complète le [Plan Qualité](#) et les [Exigences non-fonctionnelles](#).

14.2 2. Périmètre des données

POPolitics manipule deux catégories de données distinctes :

Catégorie	Exemples	Régime juridique
Données publiques d'élus	Votes, mandats, groupes politiques, amendements	Licence Ouverte Etalab 2.0 / CC BY 4.0, pas de contrainte RGPD
Données utilisateurs	Email, mot de passe hashé, élus suivis, thèmes d'intérêt	Données personnelles soumises au RGPD

Les données d'élus sont des **données publiques** liées à l'exercice d'un mandat électif. Leur traitement est légalement autorisé sans consentement individuel. Seules les données des **utilisateurs inscrits** relèvent du RGPD.

14.3 3. Authentification & gestion des accès

14.3.1 3.1 Mécanisme d'authentification

- Authentification via **JWT** (JSON Web Token) ou session sécurisée avec cookie **HttpOnly** et flag **Secure**.
- Les tokens JWT ont une durée de vie courte : **< 1 heure** avec mécanisme de refresh token.
- Les refresh tokens sont **révocables** (invalidation côté serveur en cas de déconnexion ou de compromission).

14.3.2 3.2 Stockage des mots de passe

- Les mots de passe sont hashés avec **bcrypt** ou **argon2** avant stockage.

- Aucun mot de passe n'est stocké en clair, ni loggé, ni transmis en clair dans les logs.
- Longueur minimale imposée : **8 caractères**.

14.3.3 3.3 Gestion des rôles

Rôle	Accès
Anonyme	Lecture publique (si activée) uniquement
Utilisateur standard	Accès complet au portail collectif + page personnelle
Administrateur	Accès aux fonctions d'administration (gestion utilisateurs, monitoring)

- Les droits sont vérifiés **côté serveur** (middleware Django), jamais uniquement côté client.
- Principe du **moindre privilège** : chaque rôle n'accède qu'aux ressources dont il a besoin.

14.4 4. Sécurité des échanges

14.4.1 4.1 Transport

- Toutes les communications frontend ↔ backend transitent en **HTTPS** dès la V1 serveur.
- En environnement local (MVP), HTTP est toléré uniquement en développement.
- Certificat TLS : Let's Encrypt (gratuit) ou certificat auto-signé pour les environnements internes.

14.4.2 4.2 Protection des APIs

- **CORS** configuré explicitement : whitelist des origines autorisées (pas de * en production).
- **CSRF** : protection activée sur toutes les routes POST/PUT/DELETE Django.
- **Rate limiting** : à implémenter sur les endpoints sensibles (login, refresh token) pour limiter les attaques par force brute.

14.4.3 4.3 Protection contre les injections

- **Injection SQL** : utilisation systématique de l'ORM Django — pas de requêtes SQL construites par concaténation de chaînes.
- **XSS** : échappement automatique des données côté Next.js (React échappe par défaut) ; pas d'utilisation de dangerouslySetInnerHTML sans validation préalable.
- **Injection de commandes** : aucune commande shell construite depuis des entrées utilisateur.

14.5 5. Protection des secrets

14.5.1 5.1 Règles

- Aucun secret (clé API, mot de passe, token, DSN de base de données) ne doit être **commité dans Git**.
- Le fichier `.env` est listé dans `.gitignore` — il n'est jamais versionné.
- Un fichier `.env.example` (sans valeurs réelles) est versionné pour documenter les variables requises.

14.5.2 5.2 Gestion en production

- Les secrets sont injectés via **variables d'environnement** dans Docker Compose.
- Pour la V1 Cloud : utilisation d'un gestionnaire de secrets (ex. AWS Secrets Manager, HashiCorp Vault) selon l'infrastructure retenue.

14.6 6. Obligations RGPD

14.6.1 6.1 Base légale du traitement

Articles de référence :

Art. 6§1a RGPD : « *La personne concernée a consenti au traitement de ses données à caractère personnel pour une ou plusieurs finalités spécifiques.* »

Art. 6§1b RGPD : « *Le traitement est nécessaire à l'exécution d'un contrat auquel la personne concernée est partie ou à l'exécution de mesures précontractuelles prises à la demande de celle-ci.* »

Art. 9§2e RGPD : « *Le traitement porte sur des données à caractère personnel qui sont manifestement rendues publiques par la personne concernée.* »

Traitement	Base légale
Données d'élus (votes, mandats)	Données manifestement rendues publiques par la personne concernée (Art. 9§2e RGPD)
Compte utilisateur	Consentement explicite (Art. 6§1a RGPD)
Préférences utilisateur (élus suivis, thèmes)	Exécution du contrat / consentement (Art. 6§1b RGPD)

14.6.2 6.2 Droits des utilisateurs

Les utilisateurs inscrits disposent des droits suivants, à exercer via l'interface ou par email :

Droit	Mise en œuvre
Droit d'accès	Export des données personnelles de l'utilisateur

Droit	Mise en œuvre
Droit de rectification	Modification du profil (email, préférences) depuis le compte
Droit à l'effacement	Suppression du compte et de toutes les données associées
Droit à la portabilité	Export des préférences au format JSON
Droit d'opposition	Désinscription et suppression du compte

14.6.3 6.3 Données collectées sur les utilisateurs

Donnée	Finalité	Durée de conservation
Email	Authentification, notifications	Durée du compte + 30 jours après suppression
Mot de passe hashé	Authentification	Durée du compte
Élus suivis	Personnalisation	Durée du compte
Thèmes d'intérêt	Personnalisation	Durée du compte
Logs de connexion	Sécurité	90 jours glissants

14.6.4 6.4 Minimisation des données

- Seules les données **strictement nécessaires** à la fonctionnalité sont collectées.
- Pas de collecte d'adresse postale, numéro de téléphone ou données de géolocalisation.
- Pas de partage des données utilisateurs avec des tiers.

14.6.5 6.5 Consentement

- À l'inscription, l'utilisateur doit **accepter explicitement** les conditions d'utilisation et la politique de confidentialité (case à cocher, non pré-cochée).
- Un lien vers la politique de confidentialité est accessible depuis toutes les pages.

14.7 7. Sécurité de l'infrastructure

14.7.1 7.1 Isolation des bases de données

- La base **Auth** (utilisateurs, tokens) est **strictement séparée** des datamarts métier (AN, Sénat, UE).
- Les services Django ne peuvent accéder qu'aux bases dont ils ont explicitement besoin (principe de ségrégation).

14.7.2 7.2 Journalisation

- Les événements de sécurité sont loggés : tentatives de connexion échouées, accès refusés, erreurs 5xx.

- Les logs ne contiennent **aucune donnée sensible** (pas de mot de passe, pas de token JWT complet).
- Les logs sont conservés **90 jours** puis supprimés.

14.7.3 7.3 Mises à jour de sécurité

- Les dépendances Python (Django, DRF, etc.) et Node.js (Next.js) doivent être maintenues à jour.
- Une vérification des vulnérabilités connues est effectuée avant chaque déploiement en production (`pip audit`, `npm audit`).

14.8 8. Responsabilités

Responsabilité	Rôle
Mise en œuvre des mesures de sécurité backend	Développeur Backend
Mise en œuvre CORS / HTTPS / CSRF	Tech Lead / DevOps
Conformité RGPD & politique de confidentialité	Product Owner
Revue de sécurité avant déploiement	Tech Lead
Gestion des incidents de sécurité	Tech Lead + toute l'équipe

15 POPolitics – Stratégie de tests

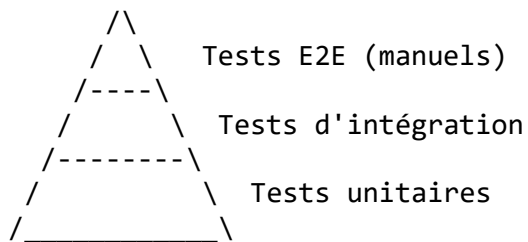
15.1 1. Introduction

Ce document décrit la stratégie de tests du projet POPolitics. Il complète le [Plan Qualité](#) et la [Definition of Done](#) en détaillant les types de tests, les outils, les responsabilités et les critères de couverture par pôle technique.

15.2 2. Principes généraux

- Tout code livré doit être accompagné des tests correspondants — c'est un critère de la Definition of Done.
 - Les tests sont **versionnés dans le dépôt Git** au même titre que le code de production.
 - On teste en priorité les **chemins critiques** (authentification, pipeline ETL, endpoints exposés au frontend).
 - Un test qui échoue **bloque le merge** de la Pull Request.
-

15.3 3. Pyramide de tests



- **Base large** : tests unitaires, rapides, nombreux.
- **Milieu** : tests d'intégration sur les interfaces entre composants.
- **Sommet** : tests E2E manuels sur les parcours utilisateurs critiques.

15.4 4. Types de tests par pôle

15.4.1 4.1 Backend (Django)

15.4.1.1 Tests unitaires

- **Outil** : pytest + pytest-django
- **Ce qu'on teste** :
 - Fonctions de validation et de transformation de données
 - Logique métier des services (Data Service, Auth Service, IA Service)
 - Serializers et permissions Django REST Framework
- **Cible de couverture** : > 80 % sur les modules critiques (MVP)

15.4.1.2 Tests d'intégration

- **Outil** : pytest-django avec base de données de test
- **Ce qu'on teste** :
 - Endpoints REST de bout en bout (requête HTTP → réponse JSON)
 - Interactions backend ↔ datamarts PostgreSQL
 - Flux d'authentification complet (inscription → login → accès protégé → déconnexion)
- **Base de données** : base PostgreSQL dédiée aux tests, réinitialisée à chaque run

15.4.1.3 Exemple de cas de test prioritaires

Cas	Type	Priorité
Login avec identifiants valides → token retourné	Intégration	Must
Login avec mauvais mot de passe → 401	Intégration	Must
GET /votes sans token → 401	Intégration	Must

Cas	Type	Priorité
GET /votes avec token valide → liste paginée	Intégration	Must
Serializer vote : champs manquants → erreur explicite	Unitaire	Should

15.4.2 4.2 Pipeline ETL (Kestra / Python)

15.4.2.1 Tests unitaires

- **Outil** : pytest
- **Ce qu'on teste** :
 - Fonctions de transformation (nettoyage, normalisation, déduplication)
 - Parsers par source (AN, Sénat, UE)
 - Validateurs de schéma (structure attendue des données Silver)

15.4.2.2 Tests de données (data quality)

- **Outil** : assertions Python dans les scripts ETL ou great_expectations (optionnel)
- **Ce qu'on teste** :
 - Nombre de lignes ingérées > 0 après un run
 - Absence de doublons sur les identifiants clés
 - Valeurs nulles dans les colonnes obligatoires
 - Cohérence des types de données (dates, entiers, chaînes)

15.4.2.3 Exemple de cas de test prioritaires

Cas	Type	Priorité
Fonction de normalisation AN : entrée JSON → sortie schéma Silver correct	Unitaire	Must
Run ETL AN : nombre de votes ingérés > 0	Data quality	Must
Cross-match : pas de doublon sur (nom, prenom, date_naissance)	Data quality	Should
Parser UE JSON-LD : champ mepId toujours présent	Unitaire	Should

15.4.3 4.3 Frontend (Next.js)

15.4.3.1 Tests unitaires

- **Outil** : Jest + React Testing Library
- **Ce qu'on teste** :

- Composants réutilisables (boutons, cartes, filtres)
- Fonctions utilitaires (formatage de dates, tri, pagination)
- Hooks personnalisés

15.4.3.2 Tests d'intégration

- **Outil** : Jest + mocks des appels API (MSW — Mock Service Worker)
- **Ce qu'on teste** :
 - Pages complètes avec données mockées (portail collectif, page personnelle)
 - Comportement des filtres et de la pagination
 - Gestion des états de chargement et d'erreur

15.4.3.3 Tests manuels (E2E)

- **Outil** : navigation manuelle dans le navigateur (Chrome / Firefox)
- **Parcours testés à chaque fin de sprint** :

Parcours	Description
Authentification	Inscription → connexion → déconnexion
Portail collectif	Consultation du tableau des votes + filtres de base
Filtres avancés	Filtrer par institution, période, groupe politique
Page personnelle	Suivre un élu → consulter sa timeline
Gestion d'erreur	Accès sans connexion → redirection login

15.5 5. Environnements de test

Environnement	Usage	Données
Local (machine développeur)	Tests unitaires et d'intégration au quotidien	Données de test synthétiques
CI (GitHub Actions)	Exécution automatique à chaque PR	Données de test synthétiques
Démo / pré-prod	Tests manuels E2E avant soutenance	Données réelles (échantillon)

15.6 6. Intégration continue (CI)

À chaque Pull Request sur la branche principale, GitHub Actions exécute automatiquement :

1. Linting du code (flake8 / ruff pour Python, ESLint pour JS)
2. Tests unitaires backend (pytest)
3. Tests unitaires frontend (jest)

4. Tests d'intégration backend (pytest-django)

Un PR ne peut pas être mergé si l'un de ces steps échoue.

15.7 7. Données de test

- Les tests utilisent des **jeux de données synthétiques** (fixtures) représentatives des données réelles.
- Les fixtures sont versionnées dans le dépôt (tests/fixtures/).
- Aucune donnée de production n'est utilisée dans les tests automatisés.
- Pour les tests manuels E2E en pré-prod, un **échantillon de données réelles** (votes AN sur 3 mois) est utilisé.

15.8 8. Responsabilités

Pôle	Responsable des tests	Type de tests
Backend	Arthur (+ relecture Jaures)	Unitaires + intégration
ETL / Data	Mehdi	Unitaires + data quality
IA	Chloé	Unitaires sur les pipelines de traitement
Frontend	Evilavy + Raphaël	Unitaires + intégration + E2E manuels
Revue globale avant soutenance	Jaures (Tech Lead)	Smoke tests sur tous les parcours

15.9 9. Critères de qualité minimaux (MVP)

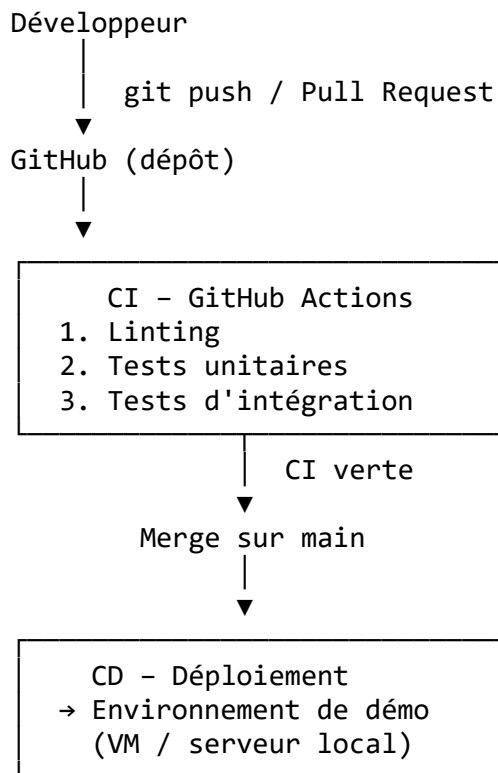
Critère	Cible
Couverture tests unitaires backend	> 80 % sur les modules critiques
Couverture tests unitaires frontend	> 50 % sur les composants partagés
Parcours E2E manuels validés	100 % des parcours Must Have
Zéro bug critique ouvert	Avant chaque démo
CI verte	Sur toutes les PR mergées

16 POPolitics – Pipeline CI/CD

16.1 1. Introduction

Ce document décrit le pipeline d'intégration continue (CI) et de déploiement continu (CD) du projet POPolitics. Il couvre les workflows automatisés, les environnements cibles et les règles de déploiement.

16.2 2. Vue d'ensemble



16.3 3. Dépôts Git

Le projet est découpé en plusieurs dépôts GitHub :

Dépôt	Contenu
popolitics/documentation	Documentation projet (ce dépôt)
popolitics/backend	Backend Django (APIs REST, Auth, IA Service)
popolitics/frontend	Frontend Next.js
popolitics/data	Pipelines ETL, scripts de transformation,

Dépôt	Contenu
	configurations Kestra

Chaque dépôt dispose de son propre pipeline CI indépendant.

16.4 4. Branches

Branche	Rôle
main	Branche stable, déployée en démo — aucun push direct
dev	Branche d'intégration — les features y sont mergées avant main
feature/<nom>	Branche de développement d'une fonctionnalité
fix/<description>	Branche de correction de bug

Règle : tout changement passe par une Pull Request. Le push direct sur main est interdit.

16.5 5. Pipeline CI (Intégration continue)

Le pipeline CI s'exécute automatiquement à chaque **push** et **Pull Request** sur dev et main.

16.5.1 5.1 Dépôt Documentation

Workflow existant : `markdown-lint.yml`

Step	Outil	Description
Markdown Lint	<code>markdownlint-cli</code>	Vérifie la syntaxe et le style de tous les fichiers <code>.md</code>

16.5.2 5.2 Dépôt Backend (Django)

Step	Outil	Description
1. Checkout	<code>actions/checkout</code>	Récupération du code
2. Setup Python	<code>actions/setup-python</code>	Installation de Python 3.11+
3. Installation des dépendances	<code>pip install -r requirements.txt</code>	Installation des packages
4. Linting	<code>ruff</code> ou <code>flake8</code>	Vérification du style PEP 8
5. Tests unitaires	<code>pytest</code>	Exécution des tests unitaires
6. Tests d'intégration	<code>pytest-django</code>	Tests avec base PostgreSQL de test

16.5.3 5.3 Dépôt Frontend (Next.js)

Step	Outil	Description
1. Checkout	actions/checkout	Récupération du code
2. Setup Node.js	actions/setup-node	Installation de Node.js 20+
3. Installation des dépendances	npm ci	Installation des packages
4. Linting	eslint	Vérification du style JS/TS
5. Tests unitaires	jest	Exécution des tests unitaires
6. Build	npm run build	Vérification que le build Next.js passe

16.5.4 5.4 Dépôt Data (ETL)

Step	Outil	Description
1. Checkout	actions/checkout	Récupération du code
2. Setup Python	actions/setup-python	Installation de Python 3.11+
3. Installation des dépendances	pip install -r requirements.txt	Installation des packages
4. Linting	ruff	Vérification du style PEP 8
5. Tests unitaires ETL	pytest	Tests des fonctions de transformation

16.6 6. Règles de merge

- Une PR ne peut être mergée que si **tous les steps CI sont verts**.
- La PR doit être approuvée par **au moins un autre membre** de l'équipe (revue de code).
- Les conflits Git doivent être résolus **par le demandeur**, pas le reviewer.

16.7 7. Pipeline CD (Déploiement continu)

16.7.1 7.1 Environnements

Environnement	Déclencheur	Infrastructure
Local (dev)	Manuel — docker-compose up	Machine du développeur
Démo / pré-prod	Merge sur main (manuel ou automatique)	VM partagée / serveur local
Production (futur)	Hors périmètre MVP	Cloud (à définir)

16.7.2 7.2 Déploiement en démo

Le déploiement sur l'environnement de démo suit les étapes suivantes :

1. Merge sur main (après CI verte + revue de code)
↓
2. Pull du code sur la VM de démo
`git pull origin main`
↓
3. Reconstruction des images Docker si nécessaire
`docker-compose build`
↓
4. Redémarrage des services
`docker-compose up -d`
↓
5. Application des migrations base de données
`docker-compose exec backend python manage.py migrate`
↓
6. Smoke tests manuels (parcours critiques)
↓
7. Démo disponible

16.7.3 7.3 Services déployés

Service	Image	Port exposé
Backend Django	popolitics/backend	8000
Frontend Next.js	popolitics/frontend	3000
PostgreSQL (Auth)	postgres:15	5432
PostgreSQL (Datamart AN)	postgres:15	5433
PostgreSQL (Datamart Sénat)	postgres:15	5434
PostgreSQL (Datamart UE)	postgres:15	5435
Kestra	kestra/kestra	8080

16.8 8. Variables d'environnement

Les variables sensibles ne sont **jamais committées** dans le dépôt. Elles sont gérées via :

- **En local** : fichier `.env` (non versionné, listé dans `.gitignore`)
- **En CI** : GitHub Actions Secrets (Settings > Secrets and variables > Actions)
- **En démo** : fichier `.env` sur la VM, accès restreint

Variables requises (exemple — voir `.env.example` dans chaque dépôt) :

```
DATABASE_URL=postgresql://user:password@localhost:5432/popolitics
SECRET_KEY=<django-secret-key>
JWT_SECRET=<jwt-secret>
KESTRA_URL=http://localhost:8080
```

16.9 9. Responsabilités

Responsabilité	Rôle
Mise en place et maintenance des workflows CI	Lazare (DevOps)
Déploiement sur la VM de démo	Lazare (DevOps)
Gestion des secrets GitHub	Lazare + Jaures (Tech Lead)
Validation post-déploiement (smoke tests)	Jaures (Tech Lead)

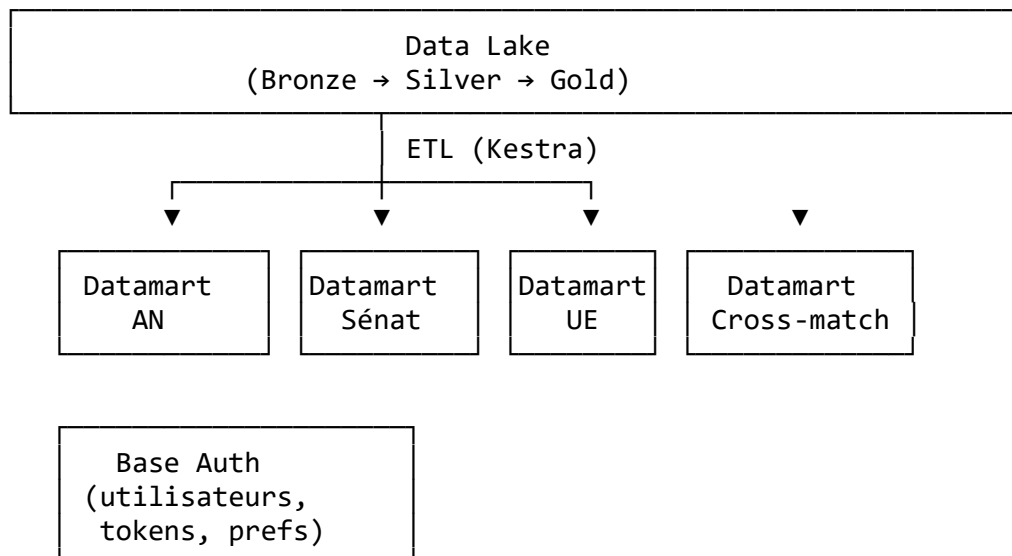
17 POPolitics – Modèle de données

17.1 1. Introduction

Ce document décrit le modèle de données de POPolitics : les entités principales, leurs attributs et leurs relations, répartis entre les 4 datamarts PostgreSQL et la base d'authentification.

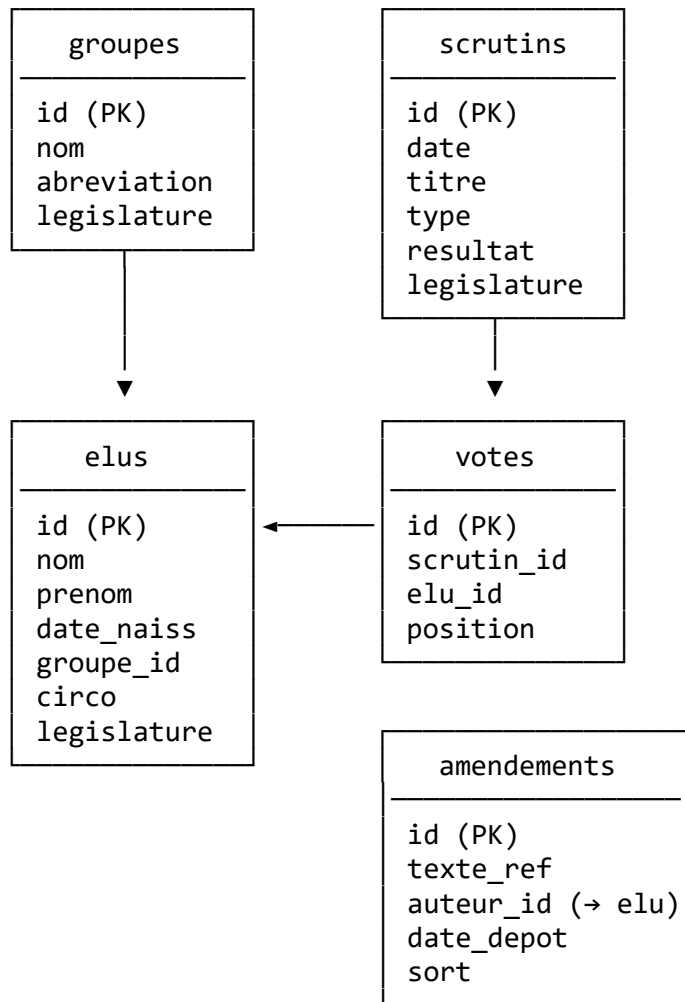
Il s'appuie sur l'architecture définie dans [01-technological-choices.md](#).

17.2 2. Vue d'ensemble des bases de données



17.3 3. Datamart Assemblée nationale

17.3.1 Schéma



17.3.2 Tables

groupes

Colonne	Type	Description
id	VARCHAR	Identifiant officiel AN
nom	VARCHAR	Nom complet du groupe
abreviation	VARCHAR	Sigle (ex. RN, RE, LFI)
legislature	INTEGER	Numéro de législature

élus

Colonne	Type	Description
id	VARCHAR	Identifiant officiel AN (acteur_uid)
nom	VARCHAR	Nom de famille
prenom	VARCHAR	Prénom
date_naissance	DATE	Date de naissance
groupe_id	VARCHAR	Référence groupes.id
circonscription	VARCHAR	Nom de la circonscription
legislature	INTEGER	Numéro de législature

scrutins

Colonne	Type	Description
id	VARCHAR	Identifiant officiel AN
date	DATE	Date du scrutin
titre	TEXT	Intitulé du vote
type	VARCHAR	Type (solennel, ordinaire...)
resultat	VARCHAR	adopte ou rejete
legislature	INTEGER	Numéro de législature

votes

Colonne	Type	Description
id	SERIAL	Identifiant interne
scrutin_id	VARCHAR	Référence scrutins.id
elu_id	VARCHAR	Référence élus.id
position	VARCHAR	pour, contre, abstention, absent

amendements

Colonne	Type	Description
id	VARCHAR	Identifiant officiel AN
texte_ref	VARCHAR	Référence du texte de loi
auteur_id	VARCHAR	Référence élus.id
date_depot	DATE	Date de dépôt
sort	VARCHAR	adopte, rejete, retire

17.4 4. Datamart Sénat

Structure similaire au datamart AN, avec les spécificités du Sénat.

17.4.1 Tables principales

senateurs

Colonne	Type	Description
id	VARCHAR	Identifiant officiel Sénat
nom	VARCHAR	Nom de famille
prenom	VARCHAR	Prénom
date_naissance	DATE	Date de naissance
groupe_id	VARCHAR	Référence groupes_senat.id
departement	VARCHAR	Département représenté
serie	INTEGER	Série de renouvellement (1 ou 2)

groupes_senat

Colonne	Type	Description
id	VARCHAR	Identifiant officiel Sénat
nom	VARCHAR	Nom complet
abreviation	VARCHAR	Sigle

scrutins_senat

Colonne	Type	Description
id	VARCHAR	Identifiant officiel Sénat
date	DATE	Date du scrutin
titre	TEXT	Intitulé du vote
type	VARCHAR	Type (public, commission...)
resultat	VARCHAR	adopte ou rejete

votes_senat

Colonne	Type	Description
id	SERIAL	Identifiant interne
scrutin_id	VARCHAR	Référence scrutins_senat.id
senateur_id	VARCHAR	Référence senateurs.id
position	VARCHAR	pour, contre, abstention, absent

17.5 5. Datamart Parlement européen

17.5.1 Tables principales

eurodeputes

Colonne	Type	Description
id	VARCHAR	Identifiant officiel UE (mepId)
nom	VARCHAR	Nom de famille
prenom	VARCHAR	Prénom
date_naissance	DATE	Date de naissance
pays	VARCHAR	Pays représenté
groupe_id	VARCHAR	Référence groupes_ue.id
legislature	INTEGER	Numéro de législature européenne

groupes_ue

Colonne	Type	Description
id	VARCHAR	Identifiant officiel UE
nom	VARCHAR	Nom complet (ex. PPE, S&D, Renew)
abreviation	VARCHAR	Sigle

scrutins_ue

Colonne	Type	Description
id	VARCHAR	Identifiant officiel UE
date	DATE	Date du scrutin
titre	TEXT	Intitulé
type	VARCHAR	Type de vote
resultat	VARCHAR	adopte ou rejete

votes_ue

Colonne	Type	Description
id	SERIAL	Identifiant interne
scrutin_id	VARCHAR	Référence scrutins_ue.id
eurodepute_id	VARCHAR	Référence eurodeputes.id
position	VARCHAR	pour, contre, abstention, absent

17.6 6. Datamart Cross-match

Permet de croiser les données entre les trois institutions. Alimenté depuis les couches Gold des trois autres datamarts.

17.6.1 Tables principales

elus_unifies

Colonne	Type	Description
id	SERIAL	Identifiant interne Cross-match
nom	VARCHAR	Nom normalisé
prenom	VARCHAR	Prénom normalisé
date_naissance	DATE	Date de naissance
elu_an_id	VARCHAR	Référence <code>elus.id</code> (NULL si pas de mandat AN)
senateur_id	VARCHAR	Référence <code>senateurs.id</code> (NULL si pas de mandat Sénat)
eurodepute_id	VARCHAR	Référence <code>eurodeputés.id</code> (NULL si pas de mandat UE)

correspondance_groupes

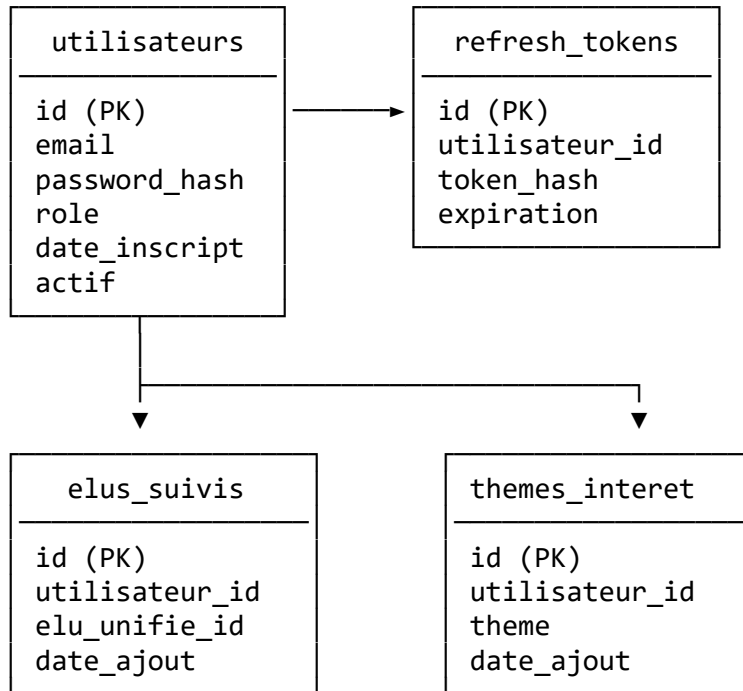
Table de mapping manuel entre les groupes des trois institutions.

Colonne	Type	Description
id	SERIAL	Identifiant interne
orientation	VARCHAR	Orientation politique (gauche, centre, droite...)
groupe_an_id	VARCHAR	Référence <code>groupes.id</code> (nullable)
groupe_senat_id	VARCHAR	Référence <code>groupes_senat.id</code> (nullable)
groupe_ue_id	VARCHAR	Référence <code>groupes_ue.id</code> (nullable)

17.7 7. Base Auth

Base strictement séparée des datamarts métier. Contient uniquement les données des utilisateurs de l'application.

17.7.1 Schéma



17.7.2 Tables

utilisateurs

Colonne	Type	Description
id	UUID	Identifiant unique utilisateur
email	VARCHAR	Email (unique)
password_hash	VARCHAR	Mot de passe hashé (bcrypt/argon2)
role	VARCHAR	user ou admin
date_inscription	TIMESTAMP	Date de création du compte
actif	BOOLEAN	Compte actif ou désactivé

refresh_tokens

Colonne	Type	Description
id	UUID	Identifiant du token
utilisateur_id	UUID	Référence utilisateurs.id
token_hash	VARCHAR	Hash du refresh token
expiration	TIMESTAMP	Date d'expiration

elus_suivis

Colonne	Type	Description
---------	------	-------------

Colonne	Type	Description
id	SERIAL	Identifiant interne
utilisateur_id	UUID	Référence utilisateurs.id
elu_unifie_id	INTEGER	Référence elus_unifies.id (Cross-match)
date_ajout	TIMESTAMP	Date de suivi
themes_interet		
Colonne	Type	Description
id	SERIAL	Identifiant interne
utilisateur_id	UUID	Référence utilisateurs.id
theme	VARCHAR	Thème choisi (ex. environnement, santé, éducation...)
date_ajout	TIMESTAMP	Date d'ajout

17.8 8. Index et performances

Table	Colonne indexée	Raison
votes	scrutin_id, elu_id	Requêtes filtrées par scrutin ou par élu
votes_senat	scrutin_id, senateur_id	Idem
votes_ue	scrutin_id, eurodepute_id	Idem
scrutins	date, legislature	Filtres temporels fréquents
elus	groupe_id	Filtres par groupe politique
elus_unifies	nom, prenom, date_naissance	Rapprochement cross-match
utilisateurs	email	Authentification (lookup unique)
elus_suivis	utilisateur_id	Chargement de la page personnelle

18 POPolitics - Diagramme de Gantt

Ce fichier centralise le planning projet sous forme de diagrammes de Gantt :

- une vue macro (sprints + jalons),
- une vue detaillee avec toutes les US planifiees par lot.

18.1 1) Vue macro

gantt

```
title POPolitics - Planning global (2026-2027)
dateFormat YYYY-MM-DD
axisFormat %d/%m/%Y
```

section Execution

```
Sprint 1 - Fondations :s1, 2026-09-21, 2026-11-20
Sprint 2 - MVP Portail collectif :s2, 2026-11-23, 2027-01-22
Sprint 3 - Enrichissements :s3, 2027-01-25, 2027-03-26
Sprint 4 - IA et finitions :s4, 2027-03-29, 2027-05-28
Sprint 5 - Stabilisation soutenance :s5, 2027-05-31, 2027-06-25
```

section Jalons

```
Lancement planning :milestone, m1, 2026-09-21, 0d
Fin Sprint 1 :milestone, m2, 2026-11-20, 0d
Fin Sprint 2 :milestone, m3, 2027-01-22, 0d
Fin Sprint 3 :milestone, m4, 2027-03-26, 0d
Fin Sprint 4 :milestone, m5, 2027-05-28, 0d
Fin de projet :milestone, m6, 2027-06-25, 0d
```

18.2 2) Vue detaillee (toutes les US)

gantt

```
title POPolitics - Gantt detaille (US par lot)
dateFormat YYYY-MM-DD
axisFormat %d/%m/%Y
```

section Cadre sprint

```
Sprint 1 :sp1, 2026-09-21, 2026-11-20
Sprint 2 :sp2, 2026-11-23, 2027-01-22
Sprint 3 :sp3, 2027-01-25, 2027-03-26
Sprint 4 :sp4, 2027-03-29, 2027-05-28
Sprint 5 :sp5, 2027-05-31, 2027-06-25
```

section Auth (auth-service)

```
0 US-101 Créer un compte :crit, us101, 2026-09-21, 2026-11-2
0 US-102 Se connecter :crit, us102, 2026-09-21, 2026-11-2
0 US-103 Se deconnecter :crit, us103, 2026-09-21, 2026-11-2
0 US-104 Reinitialiser mot de passe :us104, 2027-03-29, 2027-05-28
```

section Data ETL (data)

```
0 US-601 Ingestion AN :crit, us601, 2026-09-21, 2026-11-2
0 US-602 Ingestion Senat :crit, us602, 2026-09-21, 2026-11-2
```

0	US-603 Ingestion UE	:crit, us603, 2026-09-21, 2026-11-2
0	US-604 Normalisation Silver	:crit, us604, 2026-09-21, 2026-11-2
	US-605 Agregation Gold	:us605, 2026-11-23, 2027-01-22
	section API (api)	
2	US-201 Tableau des votes	:crit, us201, 2026-11-23, 2027-01-2
	US-204 Indicateurs d un vote	:us204, 2026-11-23, 2027-01-22
	US-301 Coherence d un élu	:us301, 2027-01-25, 2027-03-26
	US-302 Comparer élus/groupes	:us302, 2027-01-25, 2027-03-26
	US-304 Croisement AN/Senat/UE	:us304, 2027-01-25, 2027-03-26
	US-303 Cartographie alliances	:us303, 2027-03-29, 2027-05-28
	section Frontend (frontend)	
2	US-202 Filtres de base	:crit, us202, 2026-11-23, 2027-01-2
	US-203 Filtres avances	:us203, 2026-11-23, 2027-01-22
	US-501 Suivre des élus	:us501, 2027-01-25, 2027-03-26
	US-502 Timeline élu suivi	:us502, 2027-01-25, 2027-03-26
	US-503 Themes d interet	:us503, 2027-01-25, 2027-03-26
	US-504 Notifications personnalisées	:us504, 2027-01-25, 2027-03-26
	US-205 Export CSV/PDF	:us205, 2027-03-29, 2027-05-28
	section IA (ia-service)	
	US-401 Resume IA debat	:us401, 2027-03-29, 2027-05-28
	US-402 Fiche éclair texte	:us402, 2027-03-29, 2027-05-28
	section Stabilisation (Sprint 5)	
5	Tests E2E et corrections	:active, st1, 2027-05-31, 2027-06-2
	Preparation soutenance	:st2, 2027-05-31, 2027-06-25

19 POPolitics – Solutions IA & Model Training

19.1 1. Vue d'ensemble

POPolitics intègre une couche IA destinée à enrichir l'exploitation des données parlementaires issues des couches **Silver / Gold**.

Les fonctionnalités IA reposent sur un **IA Service indépendant du frontend**, consommé par le backend Django via API.

La couche IA permet :

- la génération automatique de résumés,
- l'analyse de similarité entre textes,

- un chatbot conversationnel sur les données parlementaires,
- la classification de documents,
- l'analyse de tendances.

Les modèles sont exécutés via un environnement dédié pouvant utiliser : - des modèles locaux auto-hébergés, - ou des API LLM externes.

20 2. Architecture IA

20.1 2.1 Vue logique

Datamarts PostgreSQL (Silver / Gold) | ▼ IA Service | | | ▼ ▼
 Traitements IA LLM Gateway | | ▼ ▼ Résultats enrichis LLM | ▼ PostgreSQL / Index
 vectoriel

20.2 2.2 Intégration applicative

L'IA Service est exposé comme un service indépendant.

Le backend Django consomme ses fonctionnalités via API REST.

Frontend NextJS | ▼ Backend Django | ▼ IA Service | |— Résumé automatique |—
 Similarité |— Chatbot RAG |— Classification |— Analyse

21 3. Cas d'usage IA

21.1 3.1 Résumés automatiques (MVP)

21.1.1 Objectif

Générer automatiquement un résumé compréhensible d'une loi, d'un amendement ou d'un débat parlementaire.

21.1.2 Sources

- textes de lois,
- amendements,
- comptes rendus,
- débats parlementaires.

Données utilisées depuis :

Data Gold PostgreSQL, documents nettoyés issus de la couche Silver.

21.1.3 Pipeline

Texte parlementaire | ▼ Nettoyage / découpage | ▼ LLM | ▼ Résumé structuré | ▼
Stockage résultat json

Exemple :

```
{ "objectif": "...", "mesures_principales": [ "...", "..." ], "impact": "...", "resume_court":  
  "..." }
```

21.2 3.2 Score de similarité textes (MVP)

21.2.1 Objectif

Comparer : - lois, - amendements, - déclarations de groupes, - positions politiques.

21.2.2 Approche

Utilisation d'embeddings vectoriels.

Texte | ▼ Embedding model | ▼ Vecteur | ▼ Recherche similarité | ▼ Score

21.2.3 Technologie

- Sentence Transformers (modèle français)
- Base vectorielle PostgreSQL + pgvector

21.2.4 Exemple de sortie

Loi A ↔ Amendement B

Similarité : 87%

21.3 3.3 Chatbot parlementaire (V1)

21.3.1 Objectif

Permettre une interrogation en langage naturel des données parlementaires.

21.3.2 Architecture

Approche RAG (Retrieval Augmented Generation).

Question utilisateur | ▼ Recherche vectorielle | ▼ Contexte parlementaire | ▼ LLM | ▼
Réponse sourcée

21.3.3 Sources interrogées

- lois,
- amendements,
- débats,

- votes,
- données parlementaires.

21.3.4 Capacités

- recherche documentaire,
- explication de textes,
- synthèse,
- questions/réponses.

22 4. Modèles IA

22.1 4.1 Modèles LLM

Deux modes de déploiement : Mode : Local / Usage : Développement, tests, confidentialité
 Mode: API externe / Usage : rodution, montée en charge

Exemples : - Mistral, - Qwen, - modèles compatibles OpenAI API.

22.2 4.2 Modèles d'embeddings

Utilisés pour : - recherche sémantique, - RAG, - similarité.

Pipeline :

Document | ▼ Découpage en chunks | ▼ Embedding | ▼ Index vectoriel

23 5. Stockage IA

23.1 5.1 Base vectorielle

Utilisation de pgvector pour : - embeddings, - recherche sémantique, - RAG.

Stockage : - documents - chunks - embeddings - metadata

24 6. Déploiement

24.1 6.1 Environnement

24.1.1 Développement :

Ollama + Modèle local

24.1.2 Production :

IA Service | ▼ LLM Gateway | └─ Modèle auto-hébergé ┬─ API LLM externe

25 7. Limites connues

Taille des textes parlementaires : Dépassement contexte LLM -> Découpage en chunks
Hallucinations LLM : Réponses incorrectes -> RAG + sources Coût API externe : Budget variable -> Modèles locaux Qualité des résumés : Dépend du modèle -> Validation humaine Similarité sémantique : Score approximatif -> Ajustement du modèle embedding

26 Benchmarks POPolitics

Scripts de mesure **reproductibles** servant à justifier, par des chiffres, les choix de stack documentés dans ../07-tech-stack-comparison.md.

L'objectif n'est **pas** de produire un benchmark de laboratoire parfait, mais de fournir une mesure honnête, reproductible sur la machine de l'équipe, qui illustre concrètement les écarts attendus entre les options envisagées.

Les chiffres dépendent fortement de la machine. Lancez toujours les deux côtés d'une comparaison **sur la même machine, dans la même session**, et reportez la config matérielle.

26.1 0. Démarrage rapide `bench.ps1`

Tout est piloté par un utilitaire unique à la racine, `bench.ps1` (Windows / PowerShell 7+) :

```
.\bench.ps1 setup          # installe les dépendances (venv Python + npm des 2
apps front)
.\bench.ps1 backend        # bench back : FastAPI vs Django (wrk via Docker)
.\bench.ps1 frontend      # bench front : Core Web Vitals (Lighthouse) + test
SEO
.\bench.ps1 seo            # uniquement le test SEO / crawlabilité
.\bench.ps1 all            # tout d'un coup
.\bench.ps1 help          # rappel des commandes
```

Prérequis : **Docker Desktop lancé** (back) et **Node.js + Chrome** (front). Les sections ci-dessous détaillent ce que chaque commande fait, et comment lancer les tests à la main.

26.2 1. Backend Django vs FastAPI (overhead du framework)

On compare l'overhead pur des deux frameworks à **serveur égal** : les deux apps sont servies par uvicorn (1 worker), exposent des endpoints identiques, et n'ont aucun middleware superflu.

La charge est générée par `wrk` (outil natif, lancé via Docker). On n'utilise pas de client Python maison : un client mono-process sature lui-même avant le serveur (~150 req/s) et

ne discrimine pas les frameworks. Endpoints : /json (réponse minimale), /deputes (200 objets).

26.2.1 Prérequis

- **Docker Desktop lancé** (pour wrk).
- venv Python avec les dépendances :

```
cd backend
python -m venv .venv ; .\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

26.2.2 Lancer (automatique) — recommandé

```
.\run.ps1 # 8 threads, 50 connexions, 12 s
.\run.ps1 -Connections 200 -Duration 15
```

run.ps1 démarre FastAPI (8001) puis Django (8002), les charge avec wrk sur /json et /deputes, affiche débit + latences, puis arrête tout. Une seule commande.

26.2.3 Lancer (manuel, 2 terminaux)

Terminal 1 – démarrer un serveur :

```
python -m uvicorn fastapi_app:app --host 0.0.0.0 --port 8001 --workers 1
python -m uvicorn django_app:application --host 0.0.0.0 --port 8002 --workers 1
```

Terminal 2 – charger via wrk (conteneur) :

```
docker run --rm williamyeh/wrk -t8 -c50 -d12s --latency http://host.docker.internal:8001/deputes
```

wrk affiche le débit (Requests/sec) et la distribution de latence (p50/p90/p99).

26.2.4 Mesurer les services réels

wrk accepte n'importe quelle URL : on peut viser directement les services du docker-compose (api, ia-service) une fois lancés, plutôt que les apps de démo.

```
docker run --rm williamyeh/wrk -t8 -c50 -d15s --latency http://host.docker.internal:8000/api/deputes
```

26.3 2. Frontend Next.js (SSR/SSG) vs SPA React

Deux apps témoins servent la **même page** (table de 200 députés) :

- frontend-next/ — Next.js 14, contenu pré-rendu côté serveur (SSG).
- frontend-spa/ — Vite + React 18, rendu 100 % côté client (CSR).

On mesure (a) les **Core Web Vitals** via Lighthouse et (b) la **crawlabilité / SEO** via le HTML brut. Toujours tester en mode **production** (build puis start/preview), jamais en dev.

26.3.1 Prérequis

Node.js. Lighthouse est appelé via npx. Chrome doit être installé (Lighthouse le pilote en headless).

26.3.2 a) Core Web Vitals (Lighthouse)

```
# 1. Builder puis servir Next.js (port 3000)
cd frontend-next ; npm install ; npm run build
node node_modules/next/dist/bin/next start -p 3000      # laisser tourner
# 2. Dans un autre terminal :
./frontend/lighthouse_bench.ps1 -Url "http://localhost:3000" -Runs 5 -Label "
Next.js"
```

```
# Puis la SPA (port 4173)
cd frontend-spa ; npm install ; npm run build
node node_modules/vite/bin/vite.js preview --port 4173 # laisser tourner
./frontend/lighthouse_bench.ps1 -Url "http://localhost:4173" -Runs 5 -Label "
SPA React"
```

En **localhost** (latence réseau ~0) et avec des données inlineées, la SPA peut faire jeu égal voire mieux : les avantages du SSR (réseau réel, cascade de fetch, SEO) ne s'y voient pas. Nous ne pouvons pas conclure « Next.js inutile » sur ce seul test, ce qui nous amène au point (b), décisif.

26.3.3 b) Crawlabilité / SEO (HTML brut)

Le critère qui justifie réellement Next.js pour un site public. On compare ce que reçoit un crawler **avant** d'exécuter le JS :

```
# Serveurs lancés (3000 = Next, 4173 = SPA) :
(Invoke-WebRequest http://localhost:3000 -UseBasicParsing).Content -match "Député" # → contenu présent
(Invoke-WebRequest http://localhost:4173 -UseBasicParsing).Content
# → <div id="root"></div> vide
```

Résultat type : Next.js sert ~90 ko de HTML contenant toute la table (« Député » ×404) ; la SPA sert ~0,5 ko avec un <body> vide (« Député » ×1, dans le <title>).

26.4 3. Où reporter les résultats

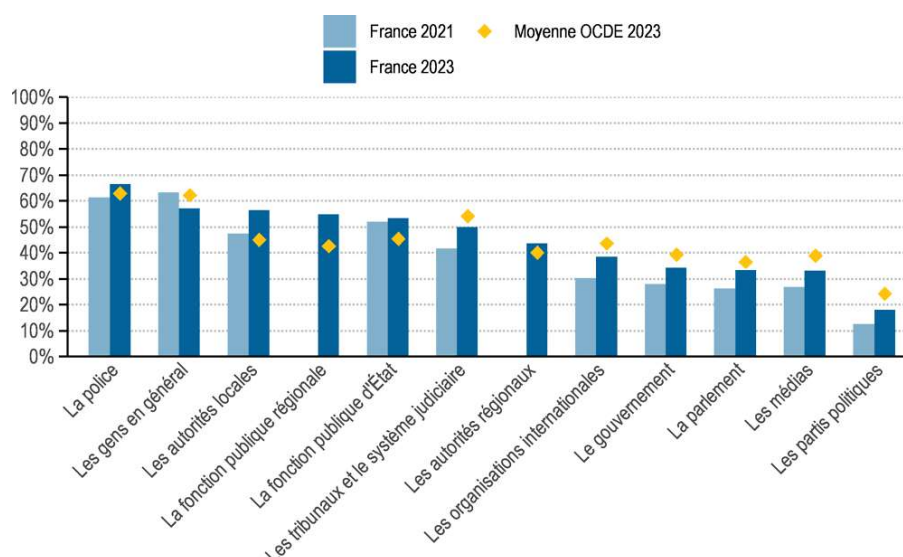
Section « **Performances mesurées** » de ../07-tech-stack-comparison.md. Compléter les tableaux et indiquer la machine de test (CPU, RAM, OS) ainsi que la date.

1. POURQUOI POPOLITICS ? — UN PROJET CIVIC-TECH NE D'UNE URGENCE DE TRANSPARENCE DEMOCRATIQUE

D'abord, pourquoi POPolitics ? L'idée est née de discussions entre proches, ami.e.s, avec des opinions politiques parfois différentes, mais partageant très souvent un même constat : la confiance envers les actions et décisions de nos représentants politiques n'est pas vraiment au beau fixe...

Ce qui est corroboré par de nombreuses études et enquêtes : la confiance envers les institutions politiques atteint même un niveau historiquement bas. Seuls **26 %** des Français déclarent faire confiance à la politique, et seulement **24 %** font confiance à l'Assemblée nationale et au Sénat (Etude CEVIPOF 2025¹). Lassitude, méfiance, morosité : trois mots qui résument l'état d'esprit politique du pays.

Les données internationales de l'OCDE² confirment aussi le diagnostic : en France, les structures parlementaires sont l'une des institutions en laquelle les citoyens ont le moins confiance.



Et à cela s'ajoute une perception alarmante de la corruption : 87 % des Français estiment qu'une majorité de responsables sont corrompus³.

Le constat est clair : le système politique manque de lisibilité, de transparence et de preuves concrètes d'intégrité.

Et pourquoi **POPolitics** alors ? Parce que c'est un savant mélange : le *poli-* de politique, le *-tics* d'analytics, et le *pop-* pour rappeler que l'information doit être accessible à tout le monde. Et puis, honnêtement... le résultat nous a bien fait rire. Et un peu de légèreté dans la politique, ça fait du bien.

¹ <https://www.sciencespo.fr/cevipof/fr/actualites/barometre-de-la-confiance-politique-du-cevipof-2025-le-grand-desarroi-democratique/>

² https://www.oecd.org/en/publications/2024/06/oecd-survey-on-drivers-of-trust-in-public-institutions-2024-results-country-notes_33192204/france_aab7c213.html

³ <https://transparency-france.org/2023/12/09/sondage-defiants-face-aux-politiques-les-francais-reclament-davantage-dexemplarite-et-de-moyens-pour-lutter-contre-la-corruption/>

2. AMBITION DU PROJET — FACILITER L'ACCES A L'INFORMATION SUR LES DECISIONS POLITIQUES

POpolitics est une application citoyenne, augmentée par la data et l'IA, qui facilite l'accès à la réalité des décisions politiques.

Son objectif : offrir au grand public un espace numérique simple, clair, moderne et transparent pour comprendre :

- comment votent les députés,
- comment évoluent leurs positions,
- quels sujets dominent le débat parlementaire,
- quelles tendances émergent dans leurs déclarations publiques,
- quels sont les réseaux d'influence et de lobbying,
- et comment se structurent réellement les forces et les prises de position politiques.

Aujourd'hui, malgré les initiatives existantes (Datan, NosDéputés) et les portails officiels Open Data⁴, aucune plateforme ne propose une **expérience unifiée, personnalisée et augmentée par l'IA** permettant de comprendre l'action politique de manière simple et immédiate.

POPolitics a donc pour ambition de devenir **un des points d'entrée incontournables** pour réintroduire **confiance** et **lisibilité** dans la relation entre citoyens et institutions démocratiques

3. FONCTIONNALITES-CLES — UNE EXPERIENCE AUGMENTEE ET CITOYENNE

Les fonctionnalités principales de l'application sont détaillées dans le doc dédié, mais nous les avons résumées ici ; elles ne sont pas définitives bien sûr, en attendant que l'équipe soit au complet !

Un portail collectif accessible et ludique

- **Historique et suivi complet des votes** : clair, fiable, mis à jour quotidiennement ; avec filtre par thème, période, groupe politique ou élu individuel. Complémentaire de Datan.fr, grâce à une présentation plus intuitive et des insights automatisés.
- **Analyse des grandes tendances politiques** : quels sujets montent, quels groupes s'opposent, quelles coalitions se forment, à l'Assemblée comme dans les commissions parlementaires
- **Synthèses automatiques des débats** : une fonctionnalité IA transforme des heures de discussions en résumés lisibles.

⁴ <https://data.assemblee-nationale.fr/> ; <https://www.assemblee-nationale.fr/>

- **Indicateurs de cohérence** des représentants : stabilité des positions, alignement au groupe, divergences notables.
- **Visualisations interactives** : réseaux politiques, comparateurs, chronologies, clusters de positionnement... et + si affinités !

Une page personnelle pour chaque citoyen

- **Notifications intelligentes personnalisées** : l'application nous prévient quand un texte majeur lié à vos centres d'intérêt arrive au vote.
- **Vue simplifiée des positions des élus que l'on suit.**
- **Fiches éclair express** : pour comprendre en quelques secondes l'essentiel d'un projet de loi, ses enjeux, ses réseaux d'influence.

POPolitics éclaire, simplifie et participe à rendre les citoyens actifs dans la vie politique

4. LES DEFIS A IDENTIFIER – ET A RELEVER

C'est bien sûr un avant-goût de ce qui nous attend, mais c'est ça qui rend le projet enthousiasmant ! Les technos utilisées pour relever ces défis seront décidées en équipe 😊

Licences et open data

L'application s'appuie sur des données publiques - pour la grande majorité ; mais nécessitera un cadre juridique robuste, notamment pour l'exploitation **des prises de parole publiques** sur les réseaux sociaux, voire dans les médias.

Volume des données et fréquence des mises à jour

Votes, interventions, déclarations sur les réseaux sociaux... tout évolue vite. POpolitics veillera à intégrer un pipeline data scalable et automatisé.

RGPD et sécurité

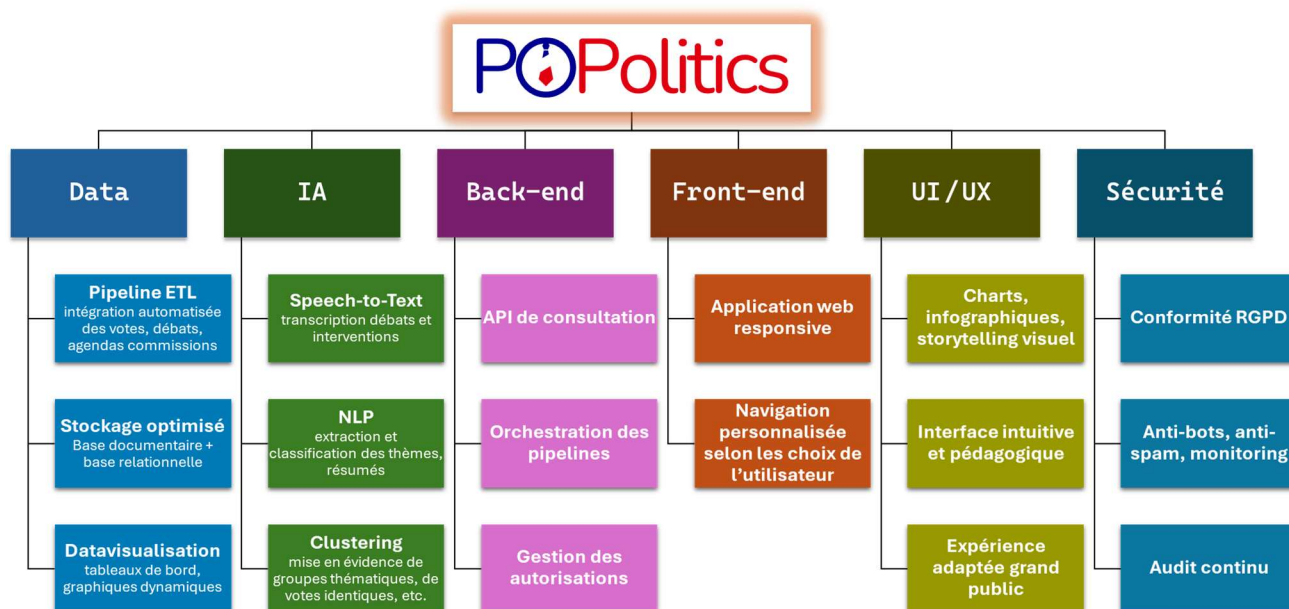
Protection des données utilisateurs, lutte contre les spams et attaques ciblées : c'est une priorité absolue.

Exigence d'auditabilité

Si on demande de la transparence et de la lisibilité, on a donc l'obligation aussi de rendre **vérifiables** et **traçables** les modèles IA, les sources utilisées, les règles d'indexation et de recommandation que nous utiliserons : ils doivent pouvoir être compris et interrogés par les utilisateurs.

POPolitics est donc un outil civic-tech : il repose sur un **socle éthique, sécurisé et transparent**.

5. ARCHITECTURE ET ORGANISATION (PBS) – 6 PILIERS FONDAMENTAUX



6. METHODOLOGIE DE PROJET — UNE ORGANISATION AGILE ET AMBITIEUSE

POPolitics sera piloté selon une méthode Agile pragmatique. L'organisation combine :

- un co-leadership : **Lead Produit & App** (expérience utilisateur, priorisation produit, go-to-market) et **Lead Data & IA** (données, qualité, modèles, pipelines). Cette double chefferie assure l'équilibre entre valeur utilisateur et faisabilité technique.
- un esprit d'équipe horizontal : les décisions stratégiques sont soumises à toute l'équipe.
- des sprints de 6 semaines : cadence retenue pour permettre des incréments significatifs (MVP fonctionnel, module IA, intégration data) tout en gardant de la flexibilité pour des éventuels feedbacks utilisateur.
- des rituels d'équipe : points d'équipe bi-mensuels (phase early), puis hebdomadaires en phase d'accélération.
- des outils de pilotage Kanban, tels que Trello, pour le flux de tâches, backlog produit centralisé (user stories + critères d'acceptation) ; voire documentation partagée (Notion / Confluence) et suivi des incidents (Sentry / issue tracker).

Équipe actuelle :

- *Mehdi (Data)* : pipelines ETL, stockage, gouvernance data.

- *Raphaël (Dev)* : architecture front & back, intégration, UI/UX fonctionnel.
- *Chloé (IA)* : modèles NLP, intégration IA, datavisualisation.

Recrutements prioritaires :

1. Architecte projet — consolide l'architecture technique, garantie évolutivité et autorisations.
2. Designer UI/UX — transforme les besoins en interfaces pédagogiques et accessibles.
3. Spécialiste cybersécurité — implémente posture de sécurité, audits et protection contre manipulation politique.

POPolitics : une équipe aux compétences complémentaires, où priment l'esprit collectif et l'envie de s'engager dans l'information citoyenne.

EN CONCLUSION — POURQUOI POPOLITICS VA COMPTER

POPolitics est un outil civic-tech, un accélérateur de compréhension, un pont entre le politique et le public, puisqu'il **transforme l'immense masse de données publiques en une information claire, lisible et personnalisée**

Face à la défiance de plus en plus massive vis-à-vis des politiques, notre projet propose :

- plus de transparence,
- plus de lisibilité,
- et donc plus de confiance !

...Et si on vise vraiment loin :

On pourra inclure l'information politique du Sénat, voire du Parlement et de la Commission européenne : toutes les déclinaisons sont possibles !

Et pourquoi pas : des outils d'éducation civique, à destination des jeunes, comme des plus grands

On veut vraiment comprendre ce que décident nos élus. En toute clarté